

Filtro de Sobel en CPU y GPU: Secuencial, NumPy, Numba y PyTorch

Materia: Sistemas Paralelos – Lic. en Sistemas, 5to año

Institución: UNTDF

Docente: Federico González Brizzio

Trabajo: Filtro de Sobel para detección de bordes

Repositorio: https://github.com/GiulianoPoeta99/tps_paralelos.git

Abstract

Este informe compara seis implementaciones del filtro de Sobel aplicadas sobre imágenes RGB de cuatro resoluciones: secuencial, NumPy, Numba paralelo CPU, Numba GPU, PyTorch CPU y PyTorch GPU. Todas las versiones siguen el mismo flujo general: conversión RGB a escala de grises mediante luminancia y aplicación de máscaras Sobel 3x3 para obtener la magnitud del gradiente. La comparación toma como base el método secuencial para calcular el speed-up de todos los métodos, tal como fue indicado en la consigna. Los resultados muestran que las salidas son consistentes entre implementaciones y que, en esta medición, Numba GPU fue el mejor método en tiempo total para todos los tamaños.

1. Introducción

El operador Sobel permite detectar bordes en una imagen a partir de cambios locales de intensidad. Para cada píxel interior se toma una vecindad de 3x3, se aplican dos máscaras de convolución, G_x y G_y , y se calcula la magnitud del gradiente. En este trabajo se utiliza la fórmula $\sqrt{G_x^2 + G_y^2}$ y se recorta el resultado al rango de intensidad $[0, 255]$.

El objetivo del trabajo es analizar cómo cambia el rendimiento al expresar el mismo algoritmo con distintos enfoques de cómputo: bucles secuenciales

en Python, vectorización con NumPy, paralelismo CPU con Numba, kernels CUDA con Numba y tensores PyTorch sobre CPU/GPU. La comparación busca mantener equivalente el criterio matemático para que las diferencias observadas correspondan al enfoque de implementación y no a cambios en el algoritmo.

2. Metodología

2.1 Entorno de ejecución

Propiedad	Valor
CPU	Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
Núcleos físicos	4
Procesadores lógicos	8
Threads por núcleo	2
RAM	15 GiB
Sistema operativo	Manjaro Linux, kernel 6.12.91-1-MANJARO
GPU	NVIDIA GeForce GTX 1650
Multiprocesadores CUDA reportados	14
Python	3.14.5
NumPy	2.4.4
Numba	0.65.1
PyTorch	2.12.0+cu130
CUDA reportado por PyTorch	13.0
Pillow	12.2.0

2.2 Implementaciones comparadas

1. **Secuencial:** implementación base con bucles Python, sin vectorización.
2. **NumPy:** implementación vectorizada mediante arreglos y operaciones por cortes.

3. Numba CPU: implementación compilada con `njit(parallel=True)` y `prange`, usando 4 workers físicos.
4. Numba GPU: implementación CUDA con un hilo por píxel y bloques de 16x16 hilos.
5. PyTorch CPU: implementación con tensores PyTorch en CPU, usando 4 workers físicos.
6. PyTorch GPU: implementación con tensores PyTorch en CUDA.

2.3 Parámetros experimentales

Parámetro	Valor
Tamaños	750x750, 1500x1500, 3000x3000, 6000x6000
Corridas por caso	5
Workers CPU paralelos	4
Fuente de resultados	final/resultados/parciales/*.csv
Métrica de salida	Porcentaje de píxeles blancos (<code>valor == 255</code>)

La carga de imágenes y el guardado de salidas quedan fuera de las mediciones. Cada fila utiliza el promedio de 5 corridas para RGB→gris, Sobel y tiempo total.

2.4 Cálculo de métricas

El método secuencial se usa como caso base para todos los speed-up:

```
speed-up = tiempo_total_secuencial / tiempo_total_metodo
```

La performance se calcula como:

```
performance (%) = speed-up / unidades_usadas * 100
```

Para Numba CPU y PyTorch CPU se usan 4 unidades, correspondientes a los núcleos físicos utilizados como workers. Para secuencial, NumPy y las versiones GPU se usa 1 unidad de comparación. En GPU no se toma 14 como divisor de performance porque los multiprocesadores CUDA no son

equivalentes a workers físicos de CPU; ese dato se reporta como característica del hardware.

3. Resultados

3.1 Imagen 750x750

Método	RGB→gris (s)	Sobel (s)	Total (s)	% blancos	Unidades
secuencial	0.138310733	0.382343170	0.520653903	0.281777778	1
numpy	0.034693624	0.006614383	0.041308007	0.281777778	1
numba_cpu	0.010843878	0.000573319	0.011417198	0.281777778	4
numba_gpu	0.001167969	0.000624656	0.001792625	0.281777778	1 comp. GPU
pytorch_cpu	0.002649721	0.006761589	0.009411310	0.281777778	4
pytorch_gpu	0.010117973	0.006472537	0.016590509	0.281777778	1 comp. GPU

Método	Speed-up vs secuencial	Performance (%)
secuencial	1.000000	100.00
numpy	12.604188	1260.42
numba_cpu	45.602599	1140.06
numba_gpu	290.442174	29044.22
pytorch_cpu	55.322150	1383.05
pytorch_gpu	31.382636	3138.26

3.2 Imagen 1500x1500

Método	RGB→gris (s)	Sobel (s)	Total (s)	% blancos	Unidades
secuencial	0.538026730	1.499926087	2.037952817	0.059955556	1
numpy	0.113359447	0.067603042	0.180962489	0.059955556	1
numba_cpu	0.004120866	0.005007883	0.009128749	0.059955556	4

Método	RGB→gris (s)	Sobel (s)	Total (s)	% blancos	Unidades
numba_gpu	0.002681795	0.002221041	0.004902836	0.059955556	1 comp. GPU
pytorch_cpu	0.021680436	0.037033428	0.058713864	0.059955556	4
pytorch_gpu	0.004482590	0.002693912	0.007176502	0.059955556	1 comp. GPU

Método	Speed-up vs secuencial	Performance (%)
secuencial	1.000000	100.00
numpy	11.261742	1126.17
numba_cpu	223.245575	5581.14
numba_gpu	415.668160	41566.82
pytorch_cpu	34.709908	867.75
pytorch_gpu	283.975789	28397.58

3.3 Imagen 3000x3000

Método	RGB→gris (s)	Sobel (s)	Total (s)	% blancos	Unidades
secuencial	2.205622351	6.289663254	8.495285605	0.001366667	1
numpy	0.738434547	0.806570722	1.545005269	0.001366667	1
numba_cpu	0.008683773	0.011001686	0.019685458	0.001366667	4
numba_gpu	0.007701786	0.007390242	0.015092028	0.001366667	1 comp. GPU
pytorch_cpu	0.059138226	0.161758470	0.220896696	0.001366667	4
pytorch_gpu	0.016800519	0.009445662	0.026246180	0.001366667	1 comp. GPU

Método	Speed-up vs secuencial	Performance (%)
secuencial	1.000000	100.00
numpy	5.498548	549.85
numba_cpu	431.551331	10788.78
numba_gpu	562.898876	56289.89

Método	Speed-up vs secuencial	Performance (%)
pytorch_cpu	38.458183	961.45
pytorch_gpu	323.677031	32367.70

3.4 Imagen 6000x6000

Método	RGB→gris (s)	Sobel (s)	Total (s)	% blancos	Unidades
secuencial	9.001387958	24.281427828	33.282815786	0.0000000000	1
numpy	2.913070440	4.490567513	7.403637953	0.0000000000	1
numba_cpu	0.028065790	0.032870512	0.060936302	0.0000000000	4
numba_gpu	0.022455198	0.026138397	0.048593595	0.0000000000	1 comp. GPU
pytorch_cpu	0.222153528	0.606586414	0.828739942	0.0000000000	4
pytorch_gpu	0.057195118	0.032686961	0.089882078	0.0000000000	1 comp. GPU

Método	Speed-up vs secuencial	Performance (%)
secuencial	1.000000	100.00
numpy	4.495468	449.55
numba_cpu	546.190279	13654.76
numba_gpu	684.921866	68492.19
pytorch_cpu	40.160748	1004.02
pytorch_gpu	370.294240	37029.42

3.5 Análisis GPU: cómputo, transferencias y comparación con CPU

3.5.1 Comparación específica: Numba GPU vs Numba CPU

En Numba GPU se separa el cómputo puro en GPU del tiempo total con transferencias. GPU cómputo suma solo los kernels RGB->gris y Sobel ; GPU con transferencias incluye además la copia CPU→GPU y GPU→CPU.

Tamaño	Numba CPU total (s)	Numba GPU cómputo (s)	Mejora GPU vs CPU	GPU con transferencias (s)
750x750	0.011417198	0.000956392	11.937781x	0.001792625
1500x1500	0.009128749	0.002765703	3.300698x	0.004902836
3000x3000	0.019685458	0.009050032	2.175181x	0.015092028
6000x6000	0.060936302	0.031016600	1.964635x	0.048593595

Tamaño	Mejora GPU vs CPU con transferencias
750x750	6.368983x
1500x1500	1.861932x
3000x3000	1.304361x
6000x6000	1.253999x

Conclusión. Numba GPU supera a Numba CPU en todos los tamaños, incluso al incluir transferencias. La mejora es muy marcada en 750x750 y se reduce en tamaños grandes porque Numba CPU también escala muy bien con 4 núcleos físicos.

3.5.2 Detalle de transferencias: Numba GPU

Tamaño	Transferencia CPU→GPU (s)	Transferencia GPU→CPU (s)	Transferencia total (s)	Cómputo GPU (s)
750x750	0.000662192	0.000174041	0.000836233	0.000956392
1500x1500	0.001496865	0.000640268	0.002137133	0.002765703
3000x3000	0.004192532	0.001849464	0.006041996	0.009050032
6000x6000	0.010809212	0.006767782	0.017576994	0.031016600

Tamaño	GPU con transferencias (s)	Speed-up vs secuencial con transferencias	Speed-up vs Numba CPU con transferencias
750x750	0.001792625	290.442174x	6.368983x
1500x1500	0.004902836	415.668160x	1.861932x
3000x3000	0.015092028	562.898876x	1.304361x

Tamaño	GPU con transferencias (s)	Speed-up vs secuencial con transferencias	Speed-up vs Numba CPU con transferencias
6000x6000	0.048593595	684.921866x	1.253999x

Conclusión. Las transferencias CPU→GPU y GPU→CPU consumen una parte relevante del total, pero no anulan la ventaja de Numba GPU. Frente al secuencial, el speed-up crece con el tamaño de imagen; frente a Numba CPU, la ventaja existe pero se vuelve más ajustada a partir de 3000x3000.

3.5.3 Comparación específica: PyTorch GPU vs PyTorch CPU

Tamaño	PyTorch CPU total (s)	PyTorch GPU cómputo (s)	Mejora GPU vs CPU	GPU con transferencias (s)
750x750	0.009411310	0.015394777	0.611331x	0.016590509
1500x1500	0.058713864	0.004065700	14.441268x	0.007176502
3000x3000	0.220896696	0.014966240	14.759665x	0.026246180
6000x6000	0.828739942	0.049590521	16.711660x	0.089882078

Tamaño	Mejora GPU vs CPU con transferencias
750x750	0.567271x
1500x1500	8.181404x
3000x3000	8.416337x
6000x6000	9.220302x

Conclusión. PyTorch GPU no conviene para 750x750 en esta medición: incluso el cómputo GPU queda por detrás de PyTorch CPU. Desde 1500x1500, la GPU sí amortiza el costo y mantiene una ventaja clara sobre PyTorch CPU.

3.5.4 Detalle de transferencias: PyTorch GPU

Tamaño	Transferencia CPU→GPU (s)	Transferencia GPU→CPU (s)	Transferencia total (s)	Cómputo GPU (s)
750x750	0.000849445	0.000346288	0.001195733	0.015394777
1500x1500	0.002550422	0.000560380	0.003110802	0.004065700

Tamaño	Transferencia CPU→GPU (s)	Transferencia GPU→CPU (s)	Transferencia total (s)	Cómputo GPU (s)
3000x3000	0.009720037	0.001559904	0.011279941	0.014966240
6000x6000	0.032163902	0.008127655	0.040291557	0.049590521

Tamaño	GPU con transferencias (s)	Speed-up vs secuencial con transferencias	Speed-up vs PyTorch CPU con transferencias
750x750	0.016590509	31.382636x	0.567271x
1500x1500	0.007176502	283.975789x	8.181404x
3000x3000	0.026246180	323.677031x	8.416337x
6000x6000	0.089882078	370.294240x	9.220302x

Conclusión. En PyTorch GPU, las transferencias pesan mucho desde 1500x1500 en adelante, pero el cómputo sobre GPU sigue siendo lo suficientemente rápido para superar a PyTorch CPU. En 750x750, el tamaño no alcanza para compensar la sobrecarga de GPU.

3.6 Comparación contra el mejor caso

En esta ejecución, el mejor tiempo total fue siempre `numba_gpu`. La tabla muestra cuántas veces más lento fue cada método respecto de ese mejor resultado para el mismo tamaño.

Tamaño	Mejor método	Mejor total (s)	NumPy / mejor	Numba CPU / mejor	PyTorch CPU / mejor	PyTorch GPU / mejor
750x750	numba_gpu	0.001792625	23.04x	6.37x	5.25x	9.25x
1500x1500	numba_gpu	0.004902836	36.91x	1.86x	11.98x	1.46x
3000x3000	numba_gpu	0.015092028	102.37x	1.30x	14.64x	1.74x
6000x6000	numba_gpu	0.048593595	152.36x	1.25x	17.05x	1.85x

3.7 Consistencia de salida

Tamaño	Secuencial % blancos	NumPy % blancos	Numba CPU % blancos	Numba GPU % blancos	PyTorch CPU % blancos
750x750	0.281777778	0.281777778	0.281777778	0.281777778	0.281777778
1500x1500	0.059955556	0.059955556	0.059955556	0.059955556	0.059955556
3000x3000	0.001366667	0.001366667	0.001366667	0.001366667	0.001366667
6000x6000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000

Tamaño	PyTorch GPU % blancos
750x750	0.281777778
1500x1500	0.059955556
3000x3000	0.001366667
6000x6000	0.000000000

Los porcentajes de píxeles blancos coinciden entre todos los métodos para cada tamaño, por lo que la comparación se centra en rendimiento y no en diferencias de salida.

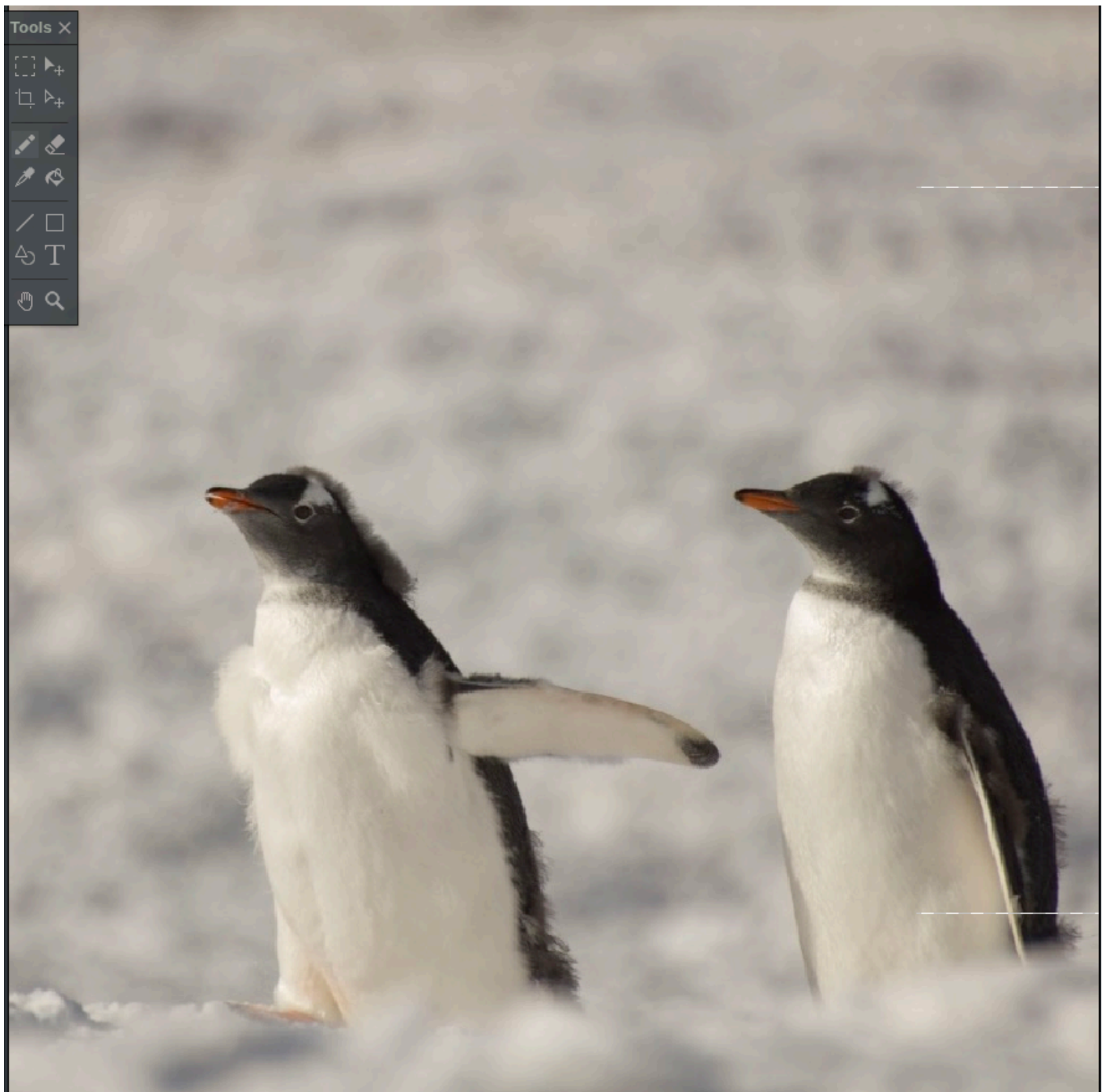
En la imagen de 6000x6000 el porcentaje de blancos es `0.000000000` para todos los métodos. Esto no significa que la imagen resultante no tenga bordes ni que exista un error de cómputo. La métrica de la consigna cuenta únicamente píxeles con valor exactamente igual a `255`; en esa resolución, ningún píxel de la salida Sobel alcanzó exactamente ese valor luego de calcular la magnitud y recortarla al rango `[0, 255]`. Los bordes pueden existir con valores menores a 255, pero no se contabilizan dentro de esta métrica puntual.

3.8 Resultados de las imágenes Antes y después del filtro Sobel

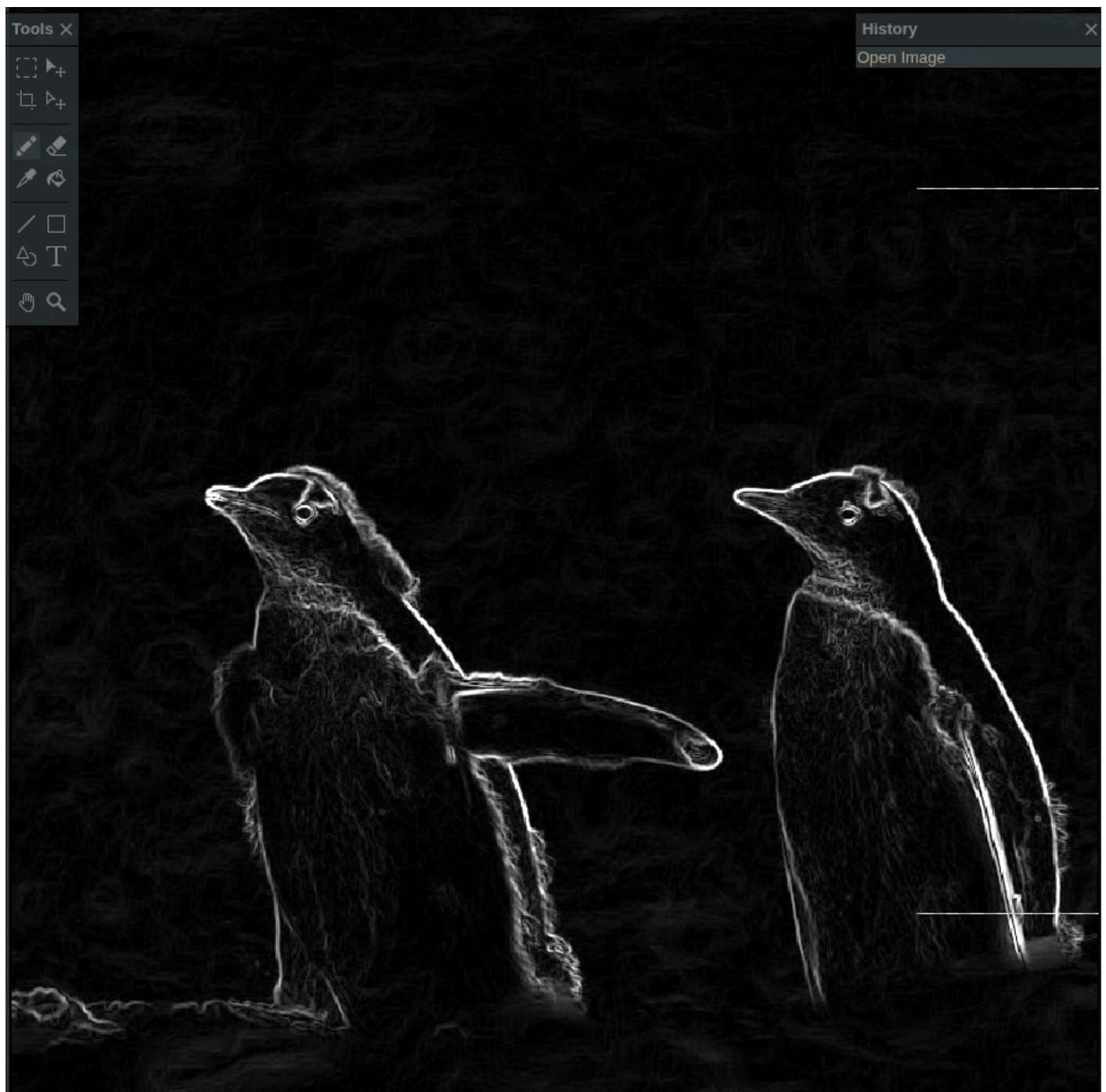
obs: las líneas blancas que aparecen son un error en el editor de imágenes.

Imagen 750x750

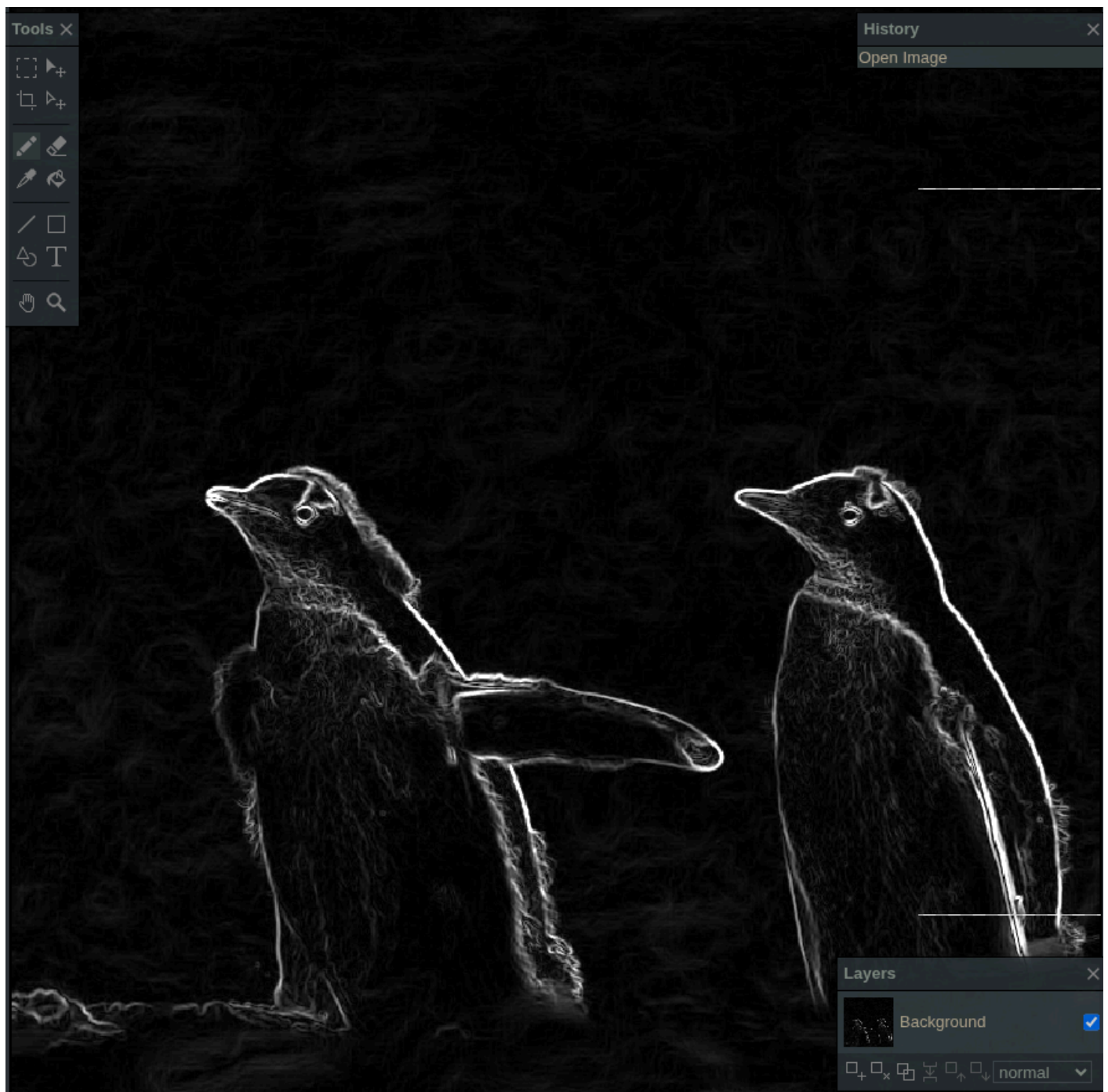
Original:



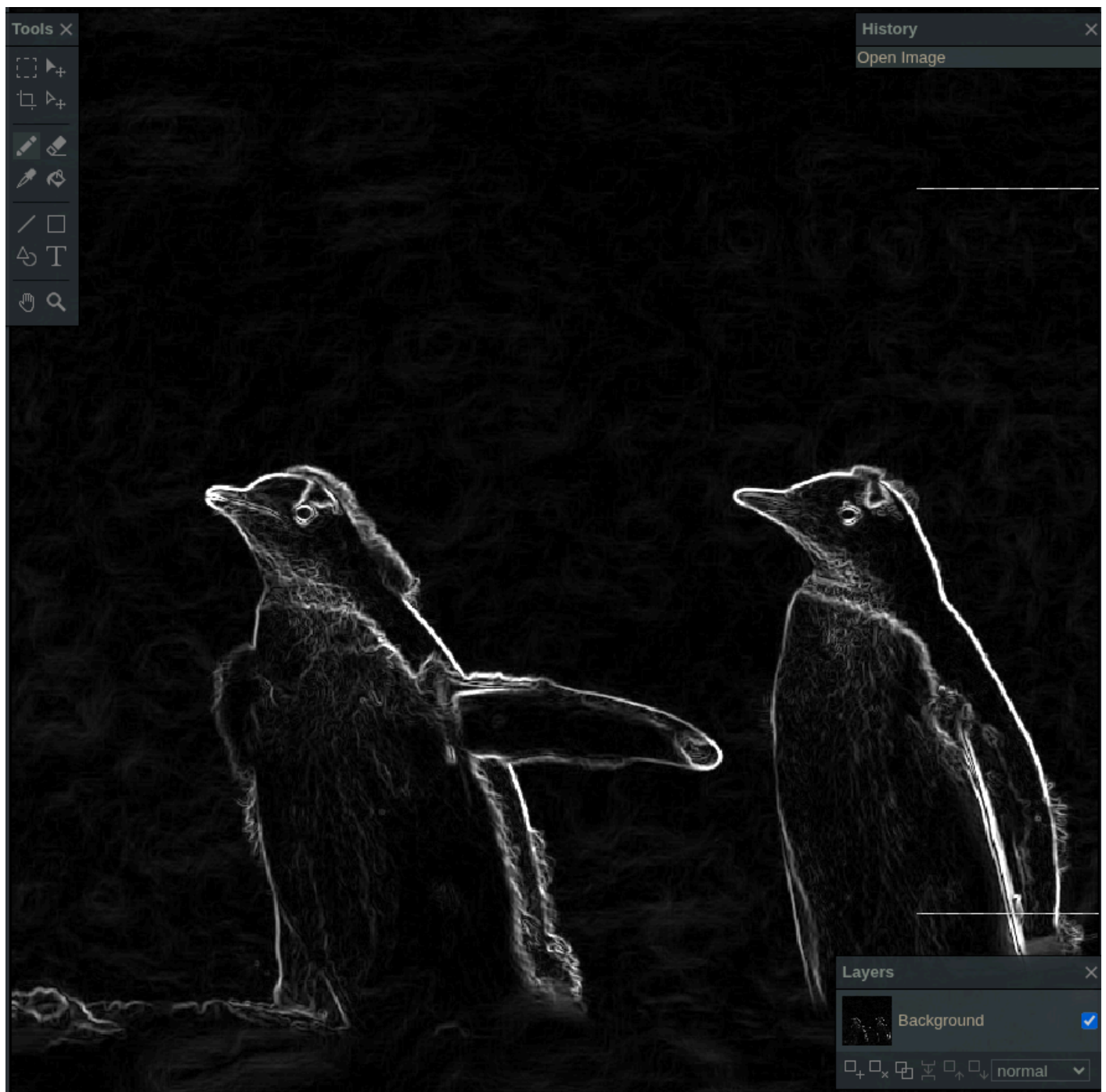
Salida Sobel secuencial:



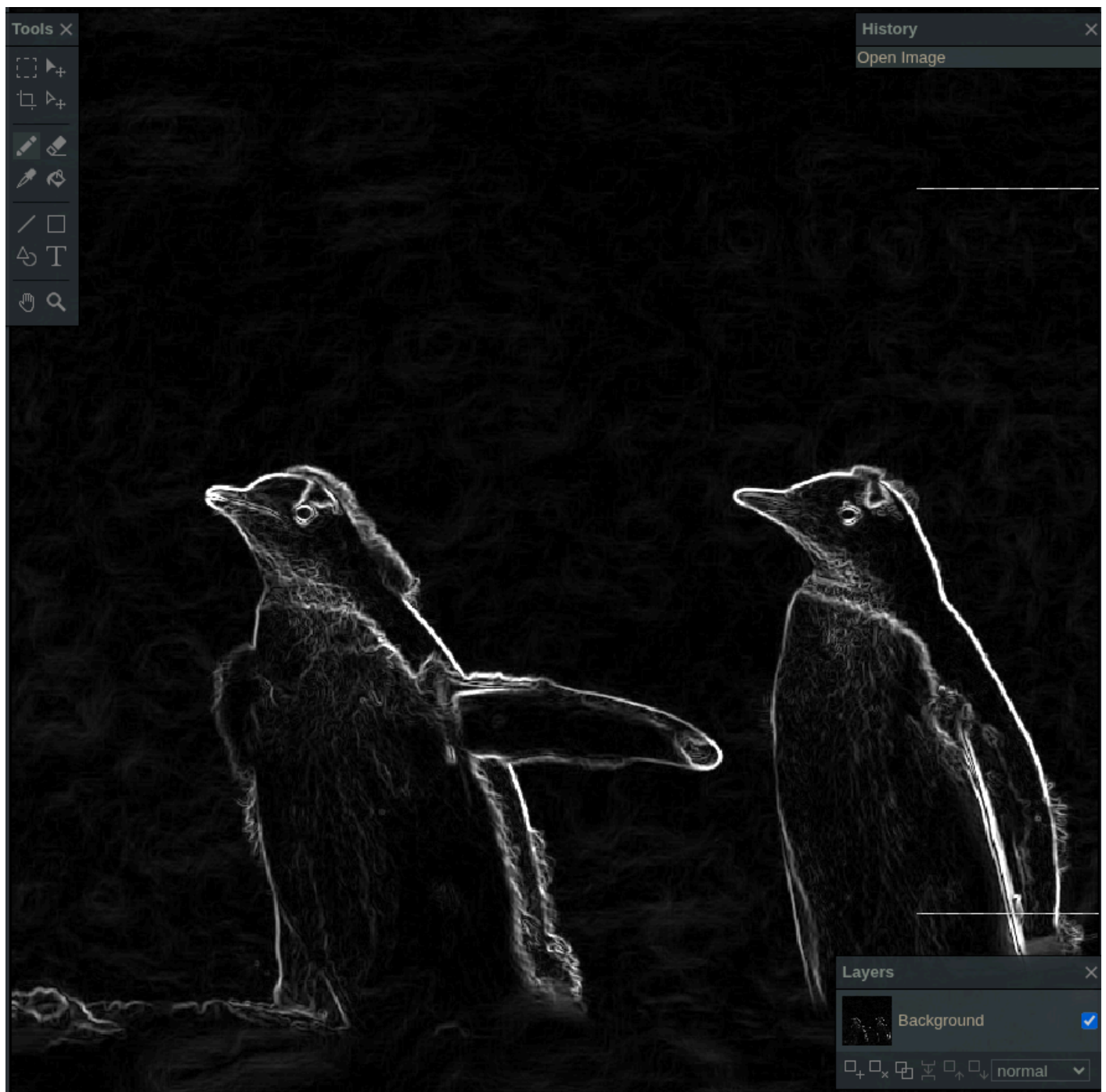
Salida Sobel NumPy:



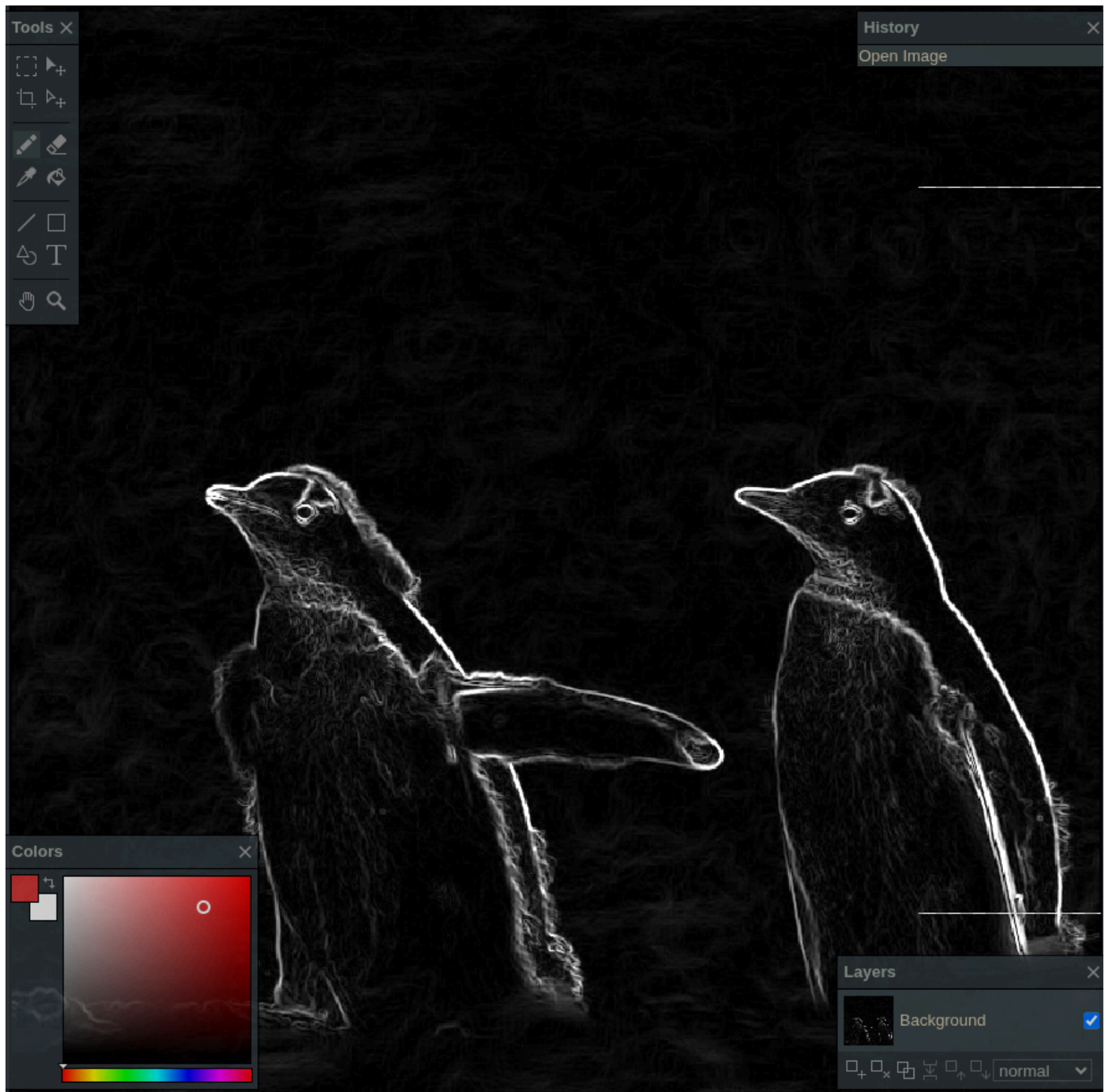
Salida Sobel Numba CPU:



Salida Sobel Numba GPU:



Salida Sobel PyTorch CPU:



Salida Sobel PyTorch GPU:

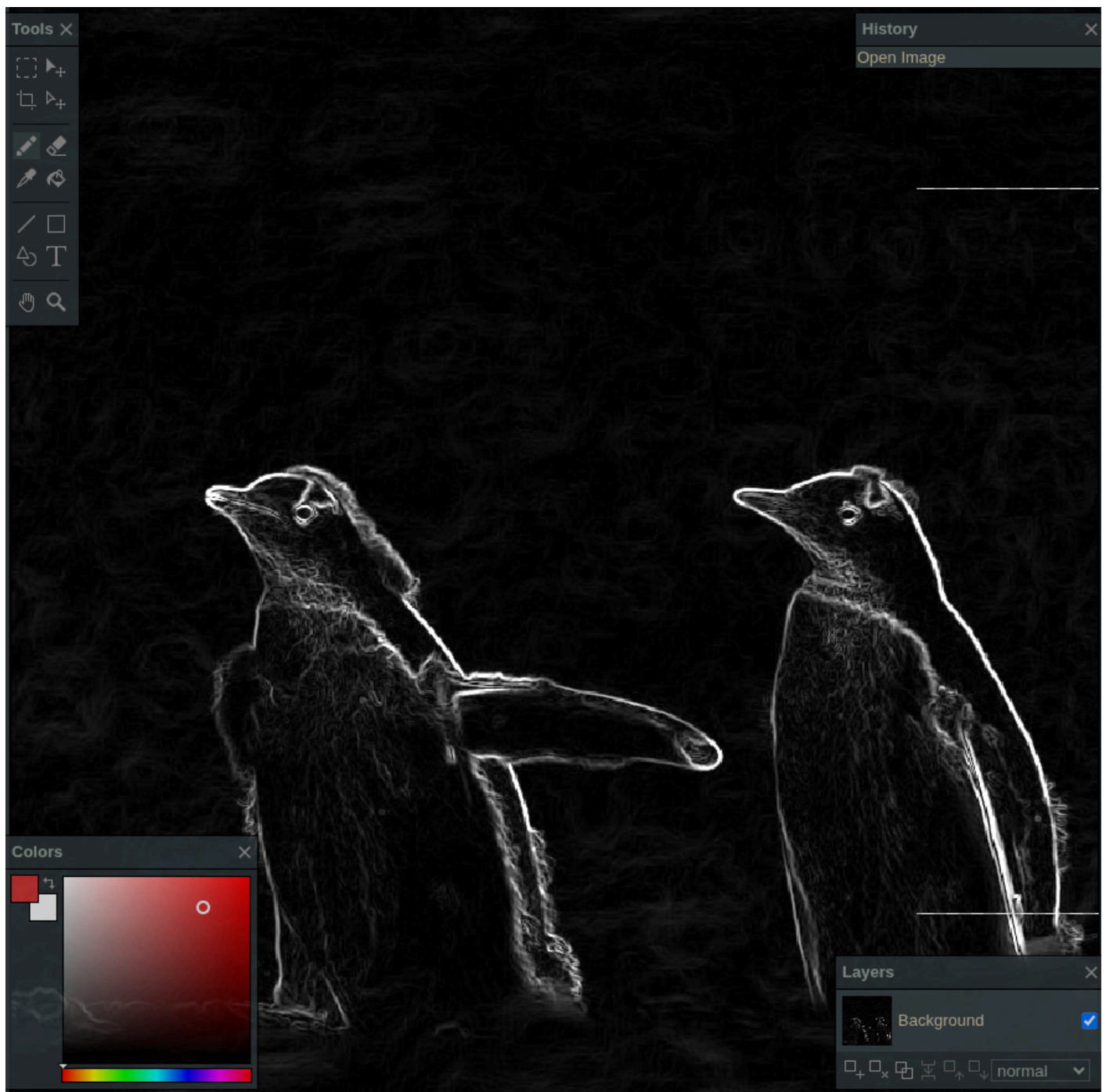
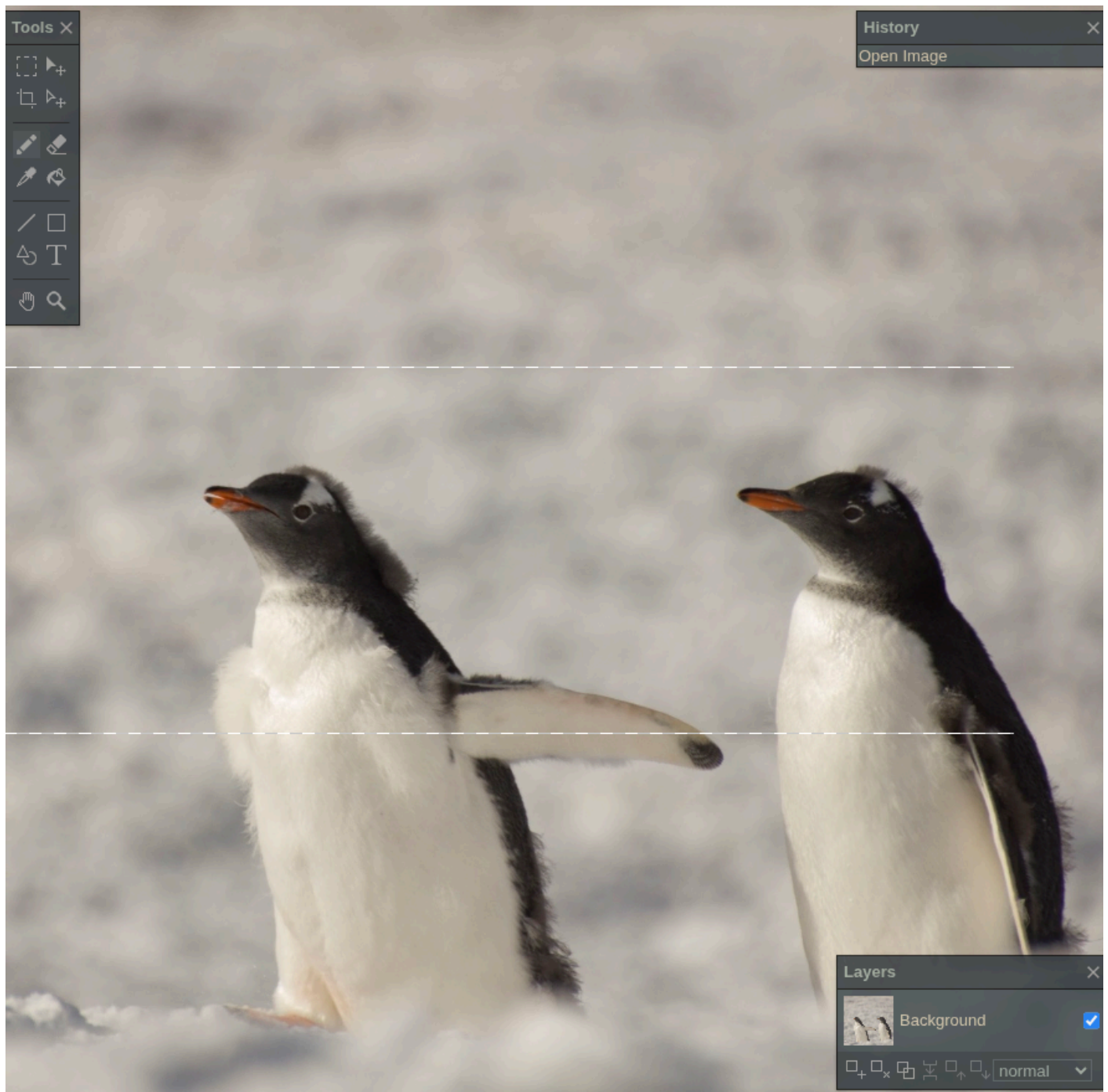
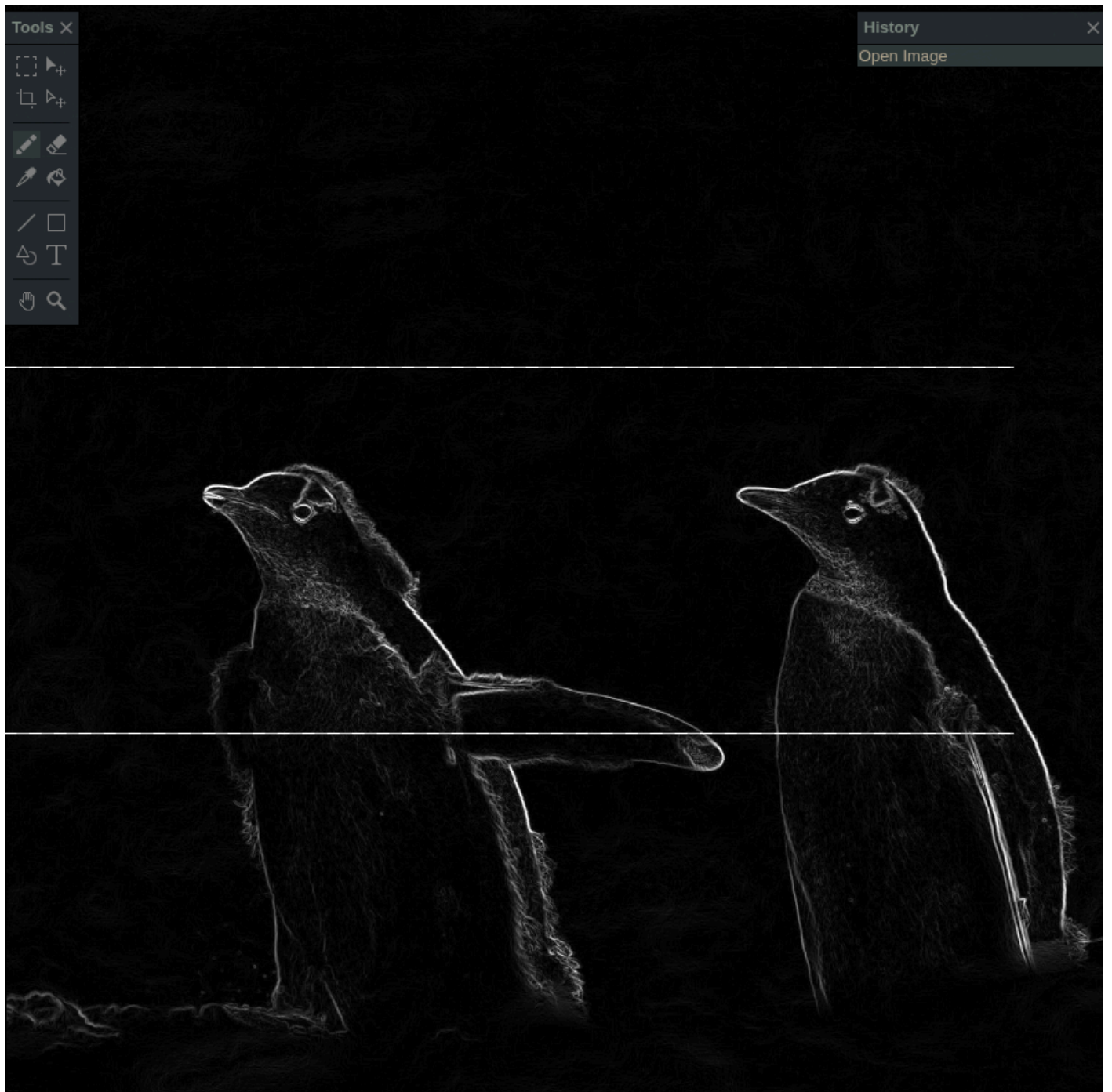


Imagen 1500x1500

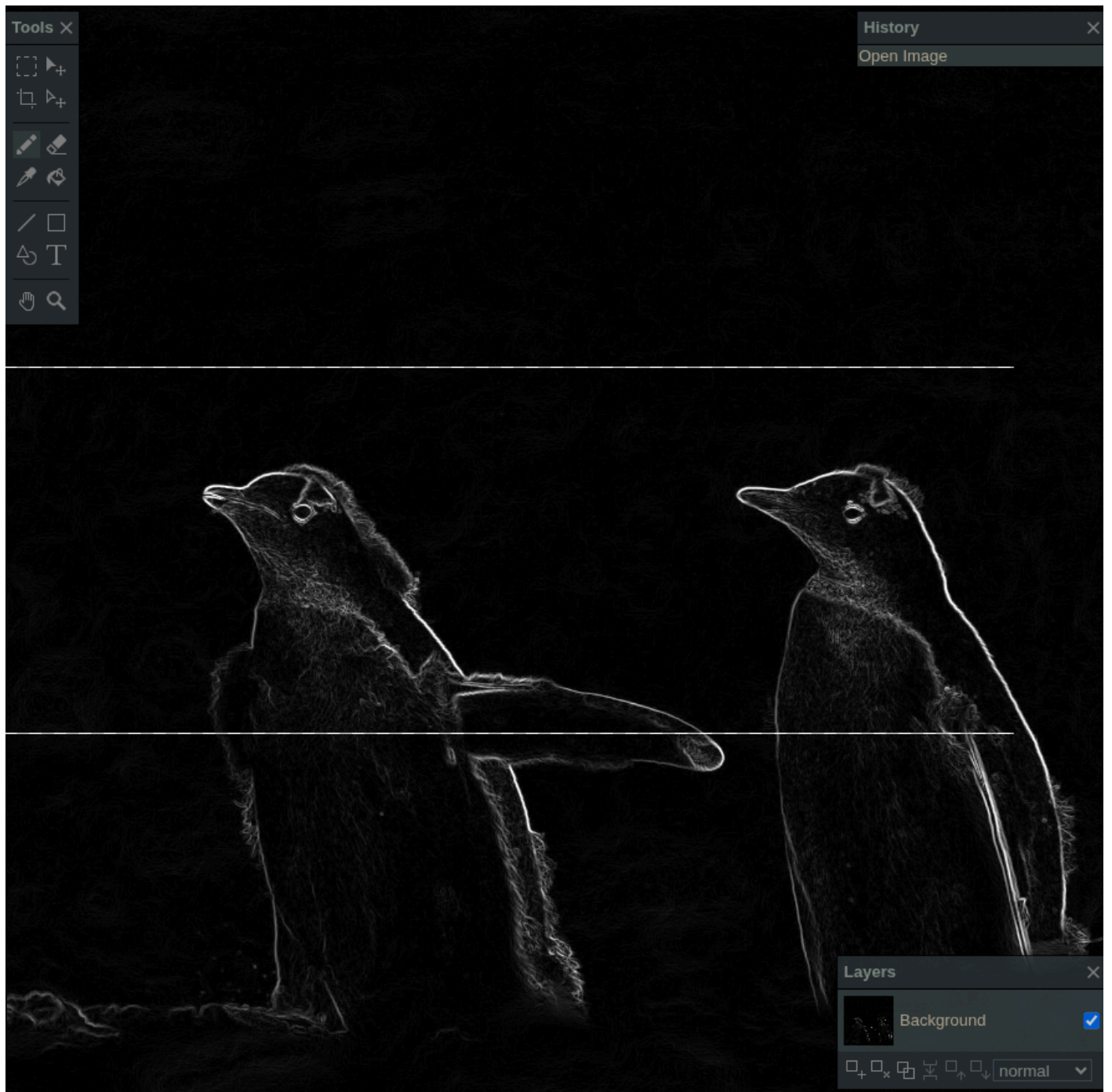
Imagen original:



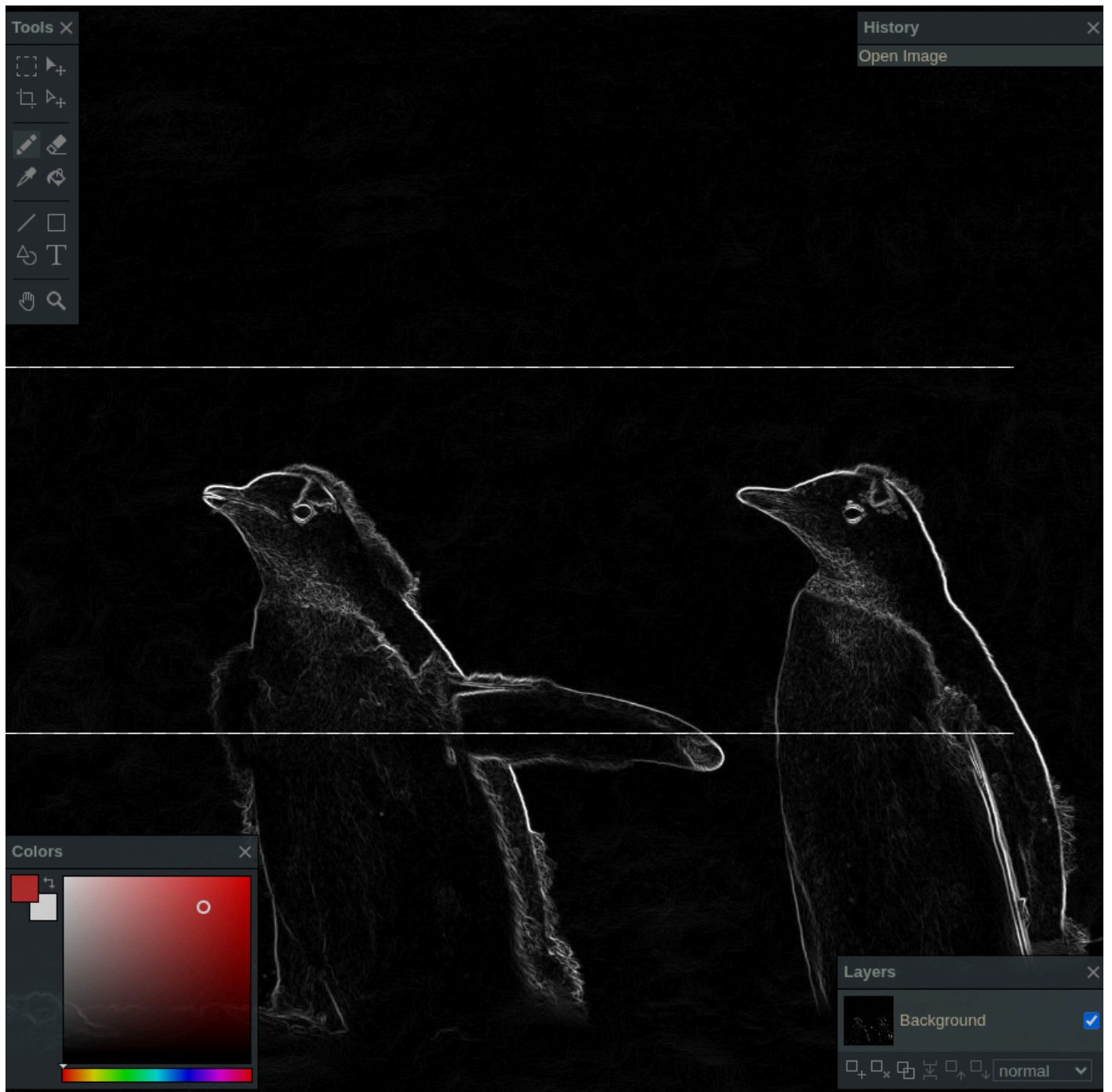
Salida Sobel secuencial:



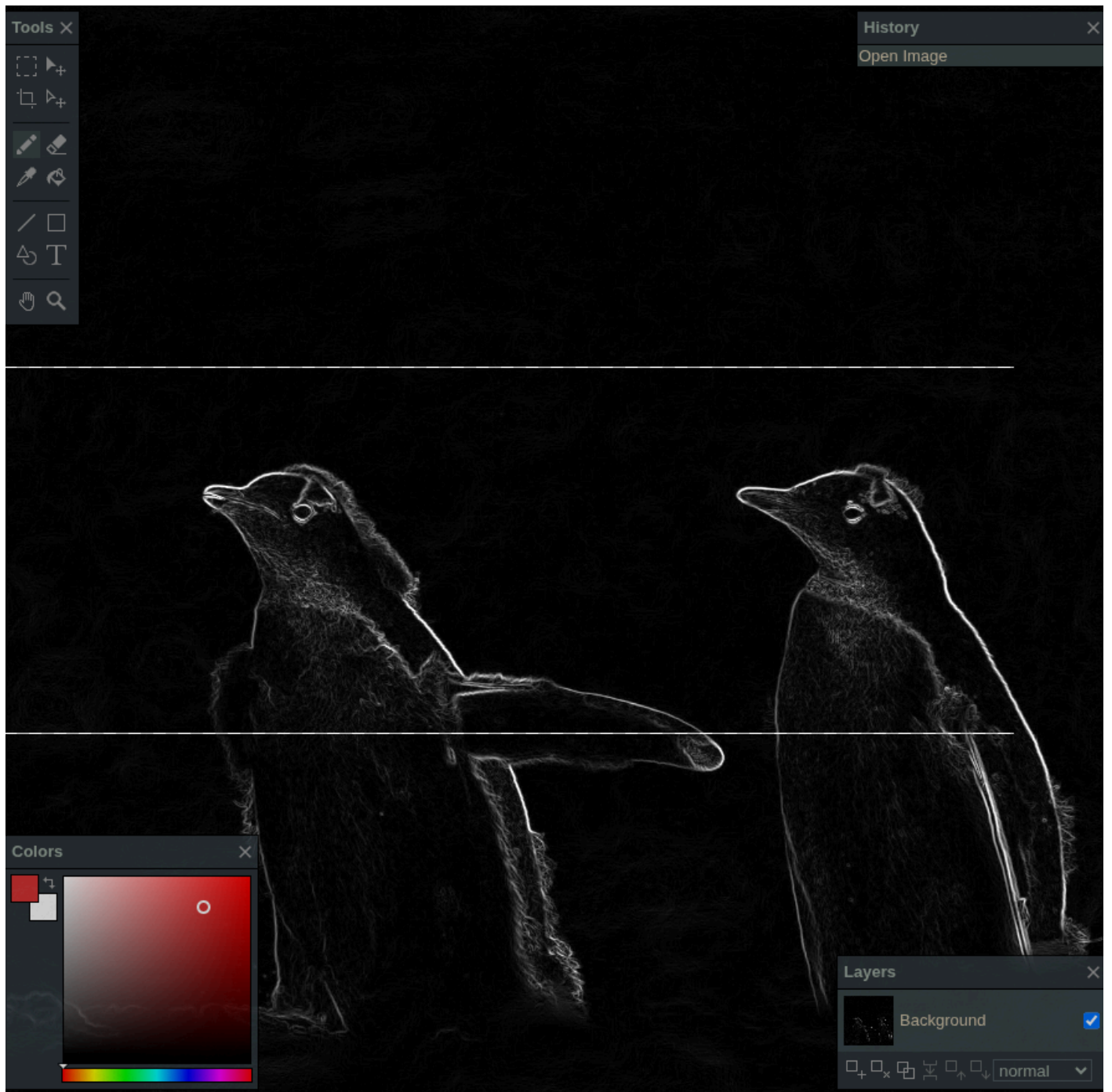
Salida Sobel NumPy:



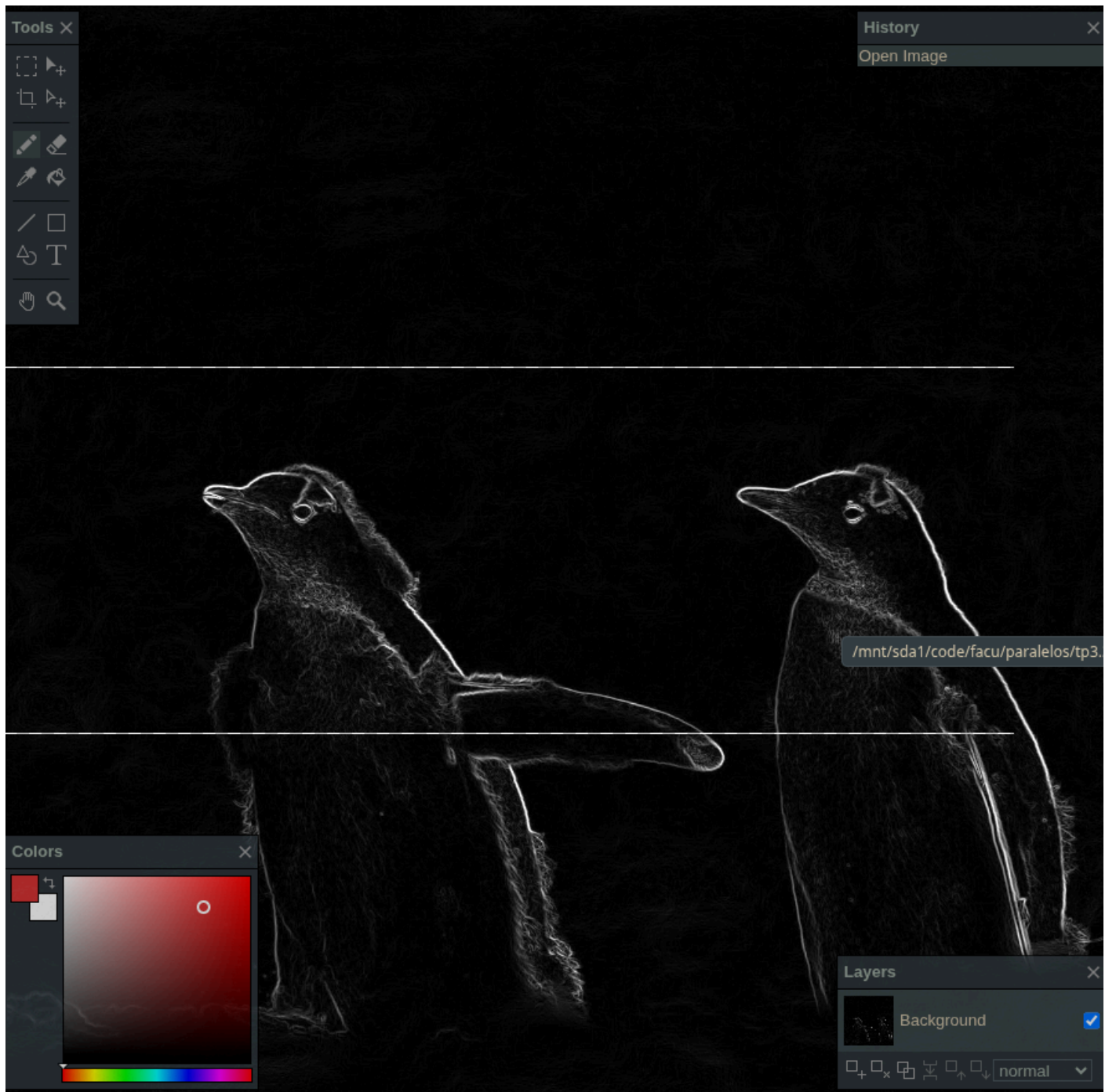
Salida Sobel Numba CPU:



Salida Sobel Numba GPU:



Salida Sobel PyTorch CPU:



Salida Sobel PyTorch GPU:

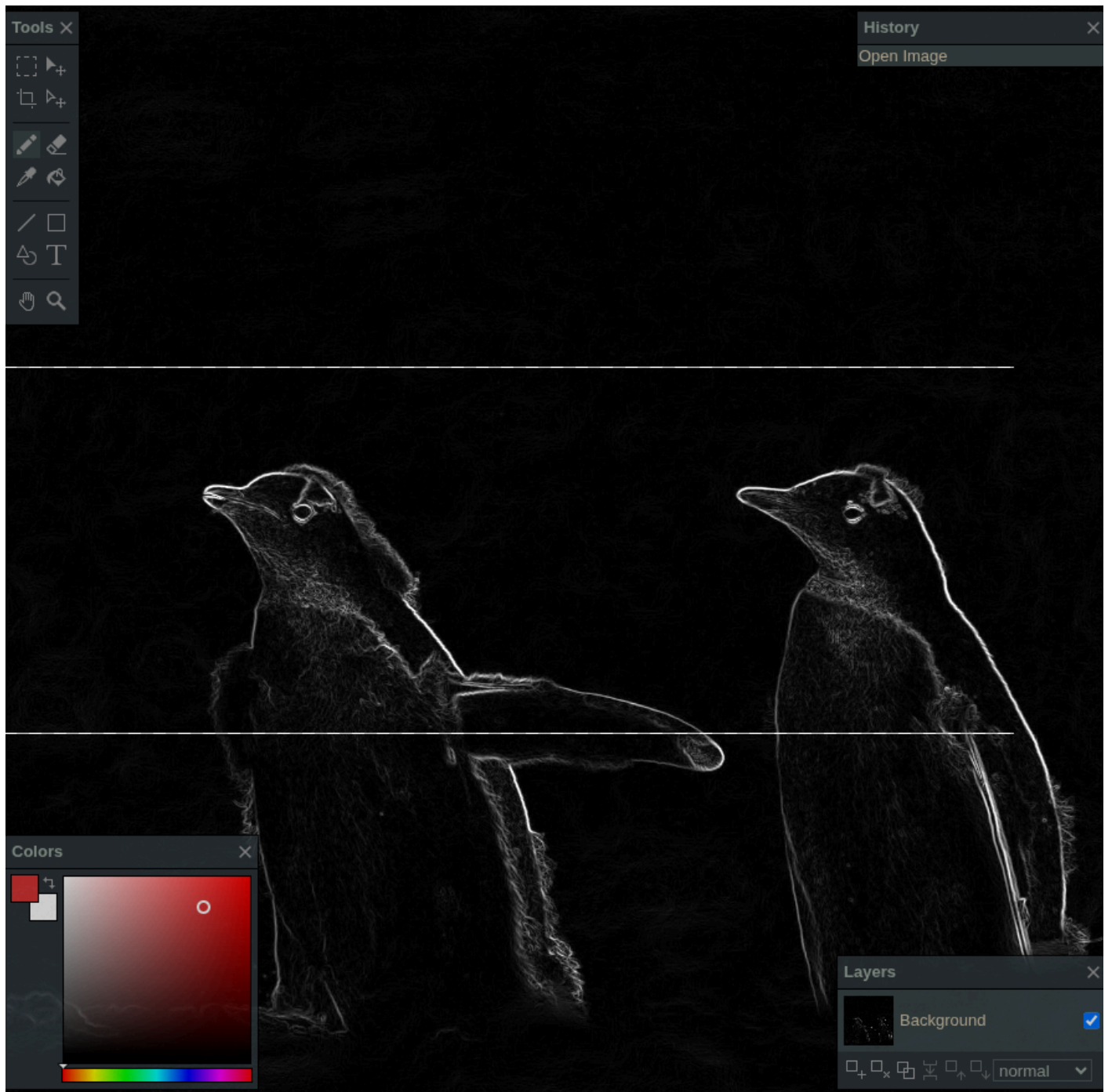
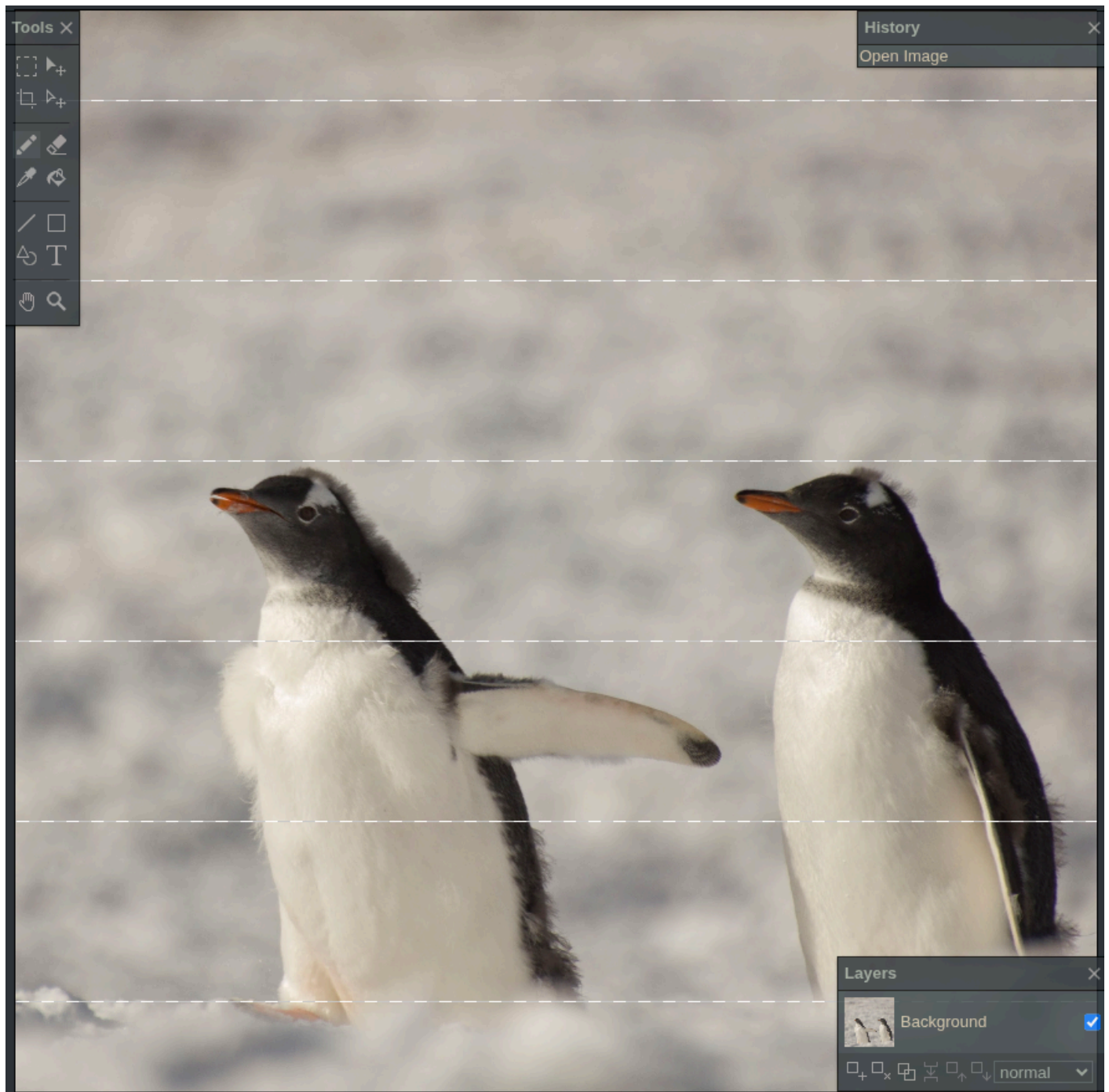


Imagen 3000x3000

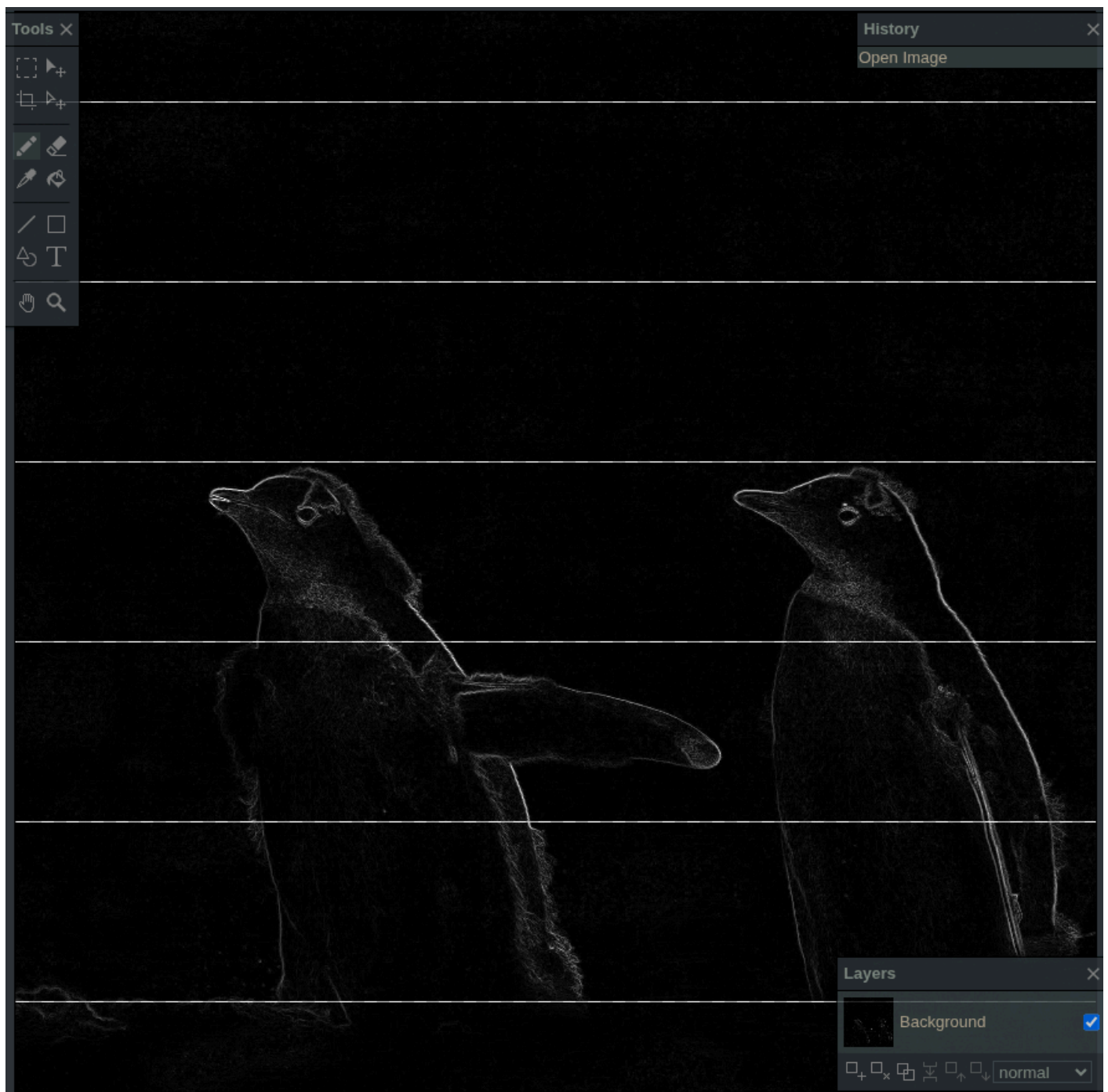
Imagen original:



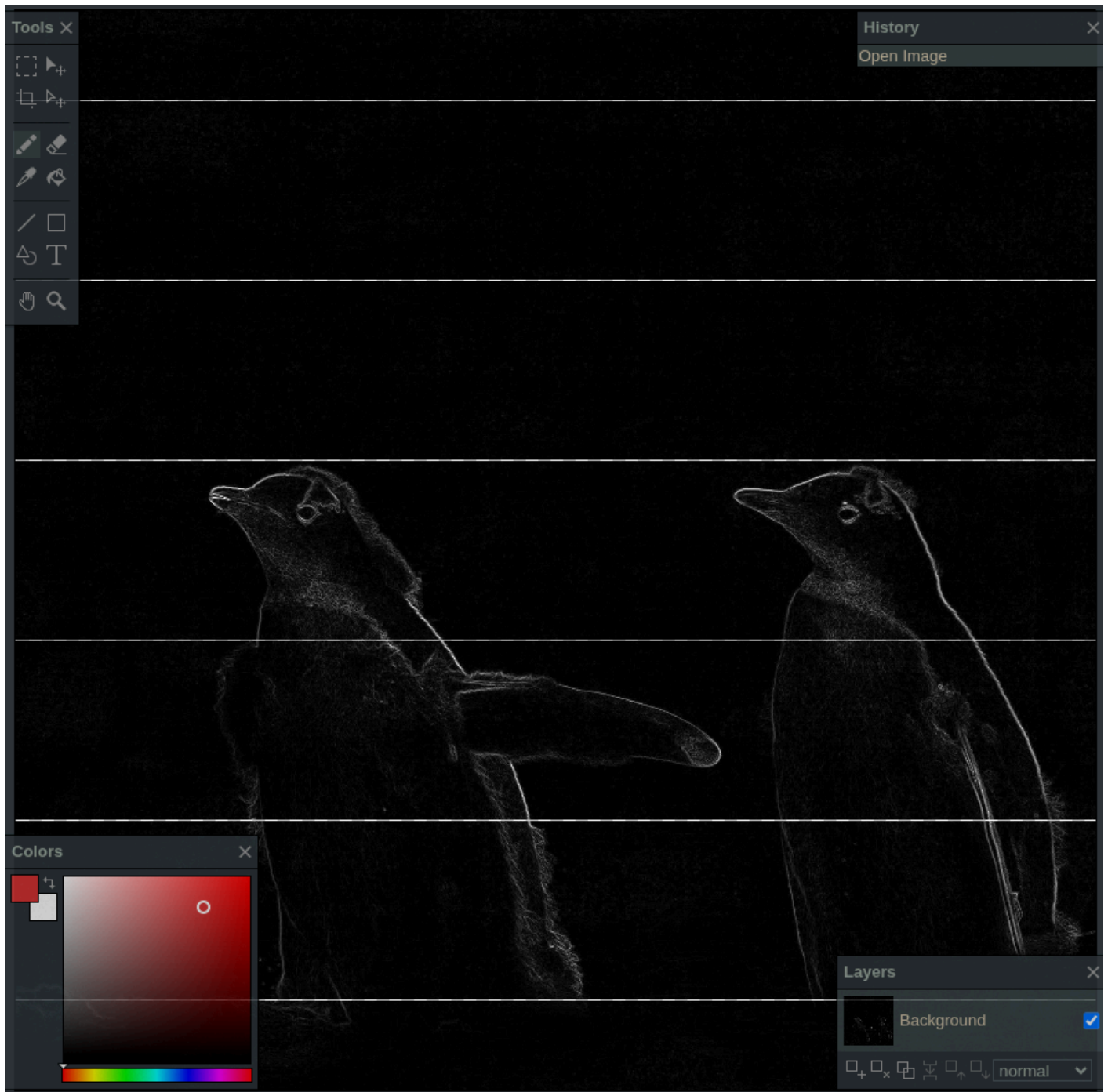
Salida Sobel secuencial:



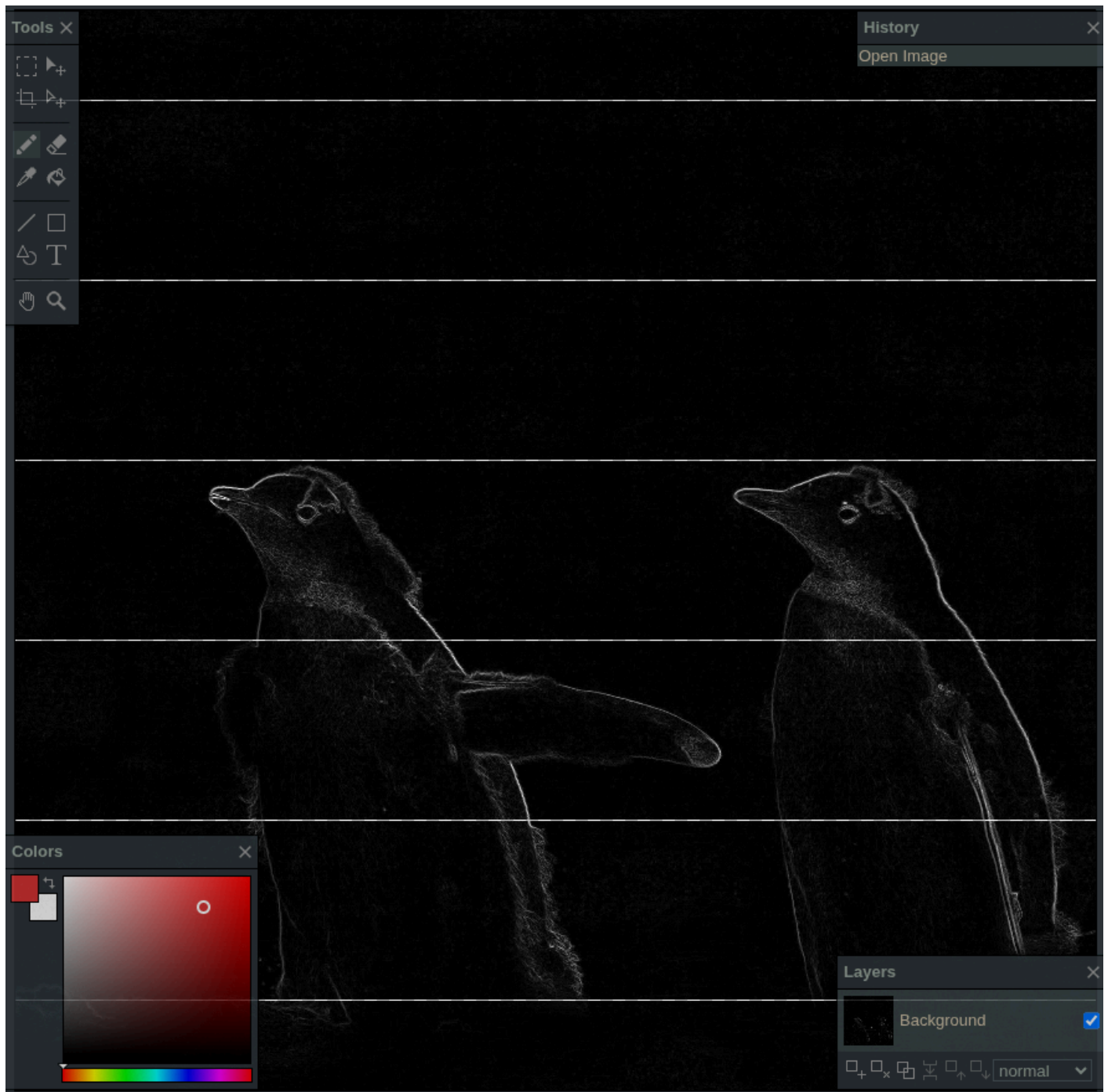
Salida Sobel NumPy:



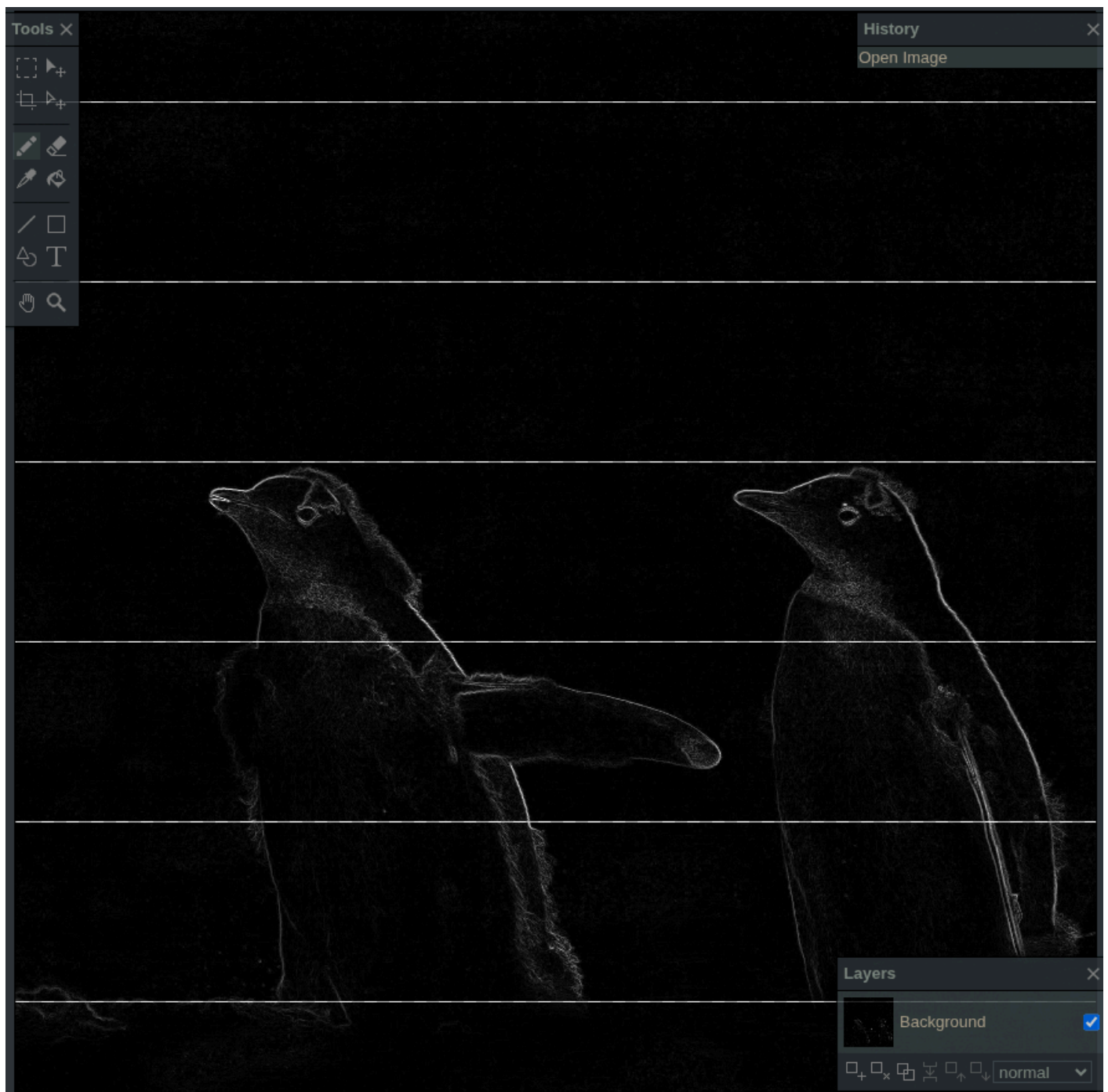
Salida Sobel Numba CPU:



Salida Sobel Numba GPU:



Salida Sobel PyTorch CPU:



Salida Sobel PyTorch GPU:

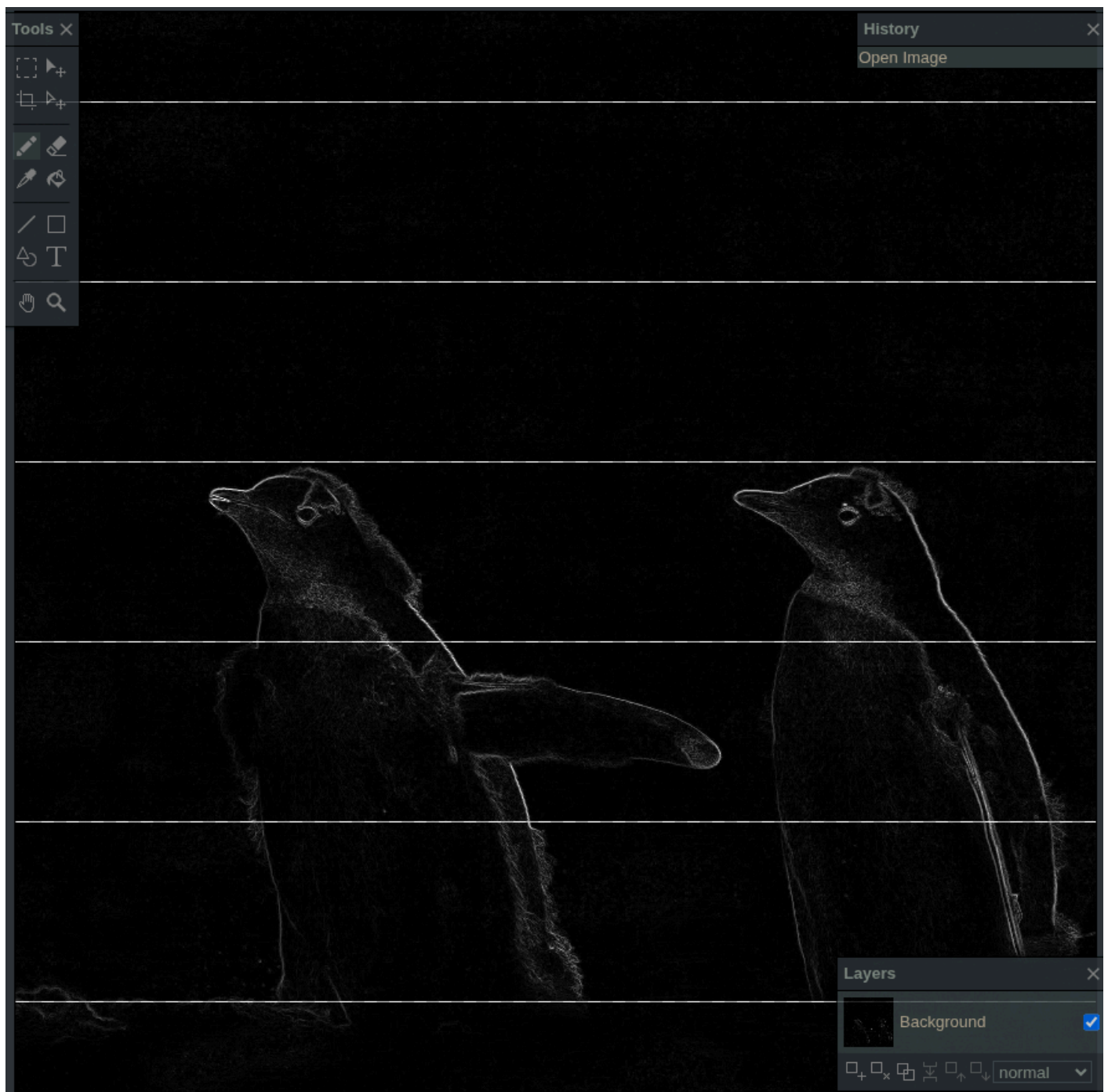
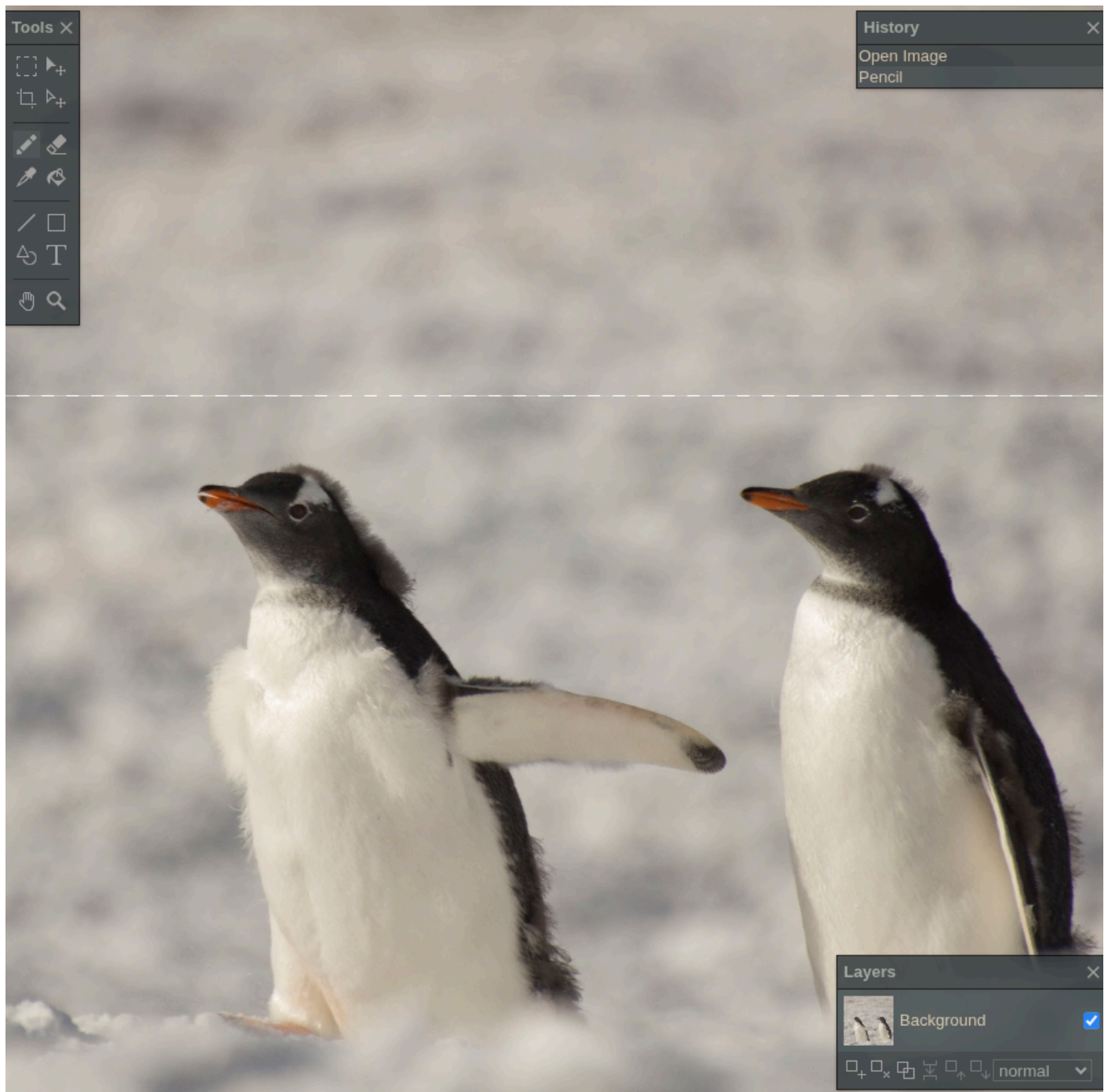
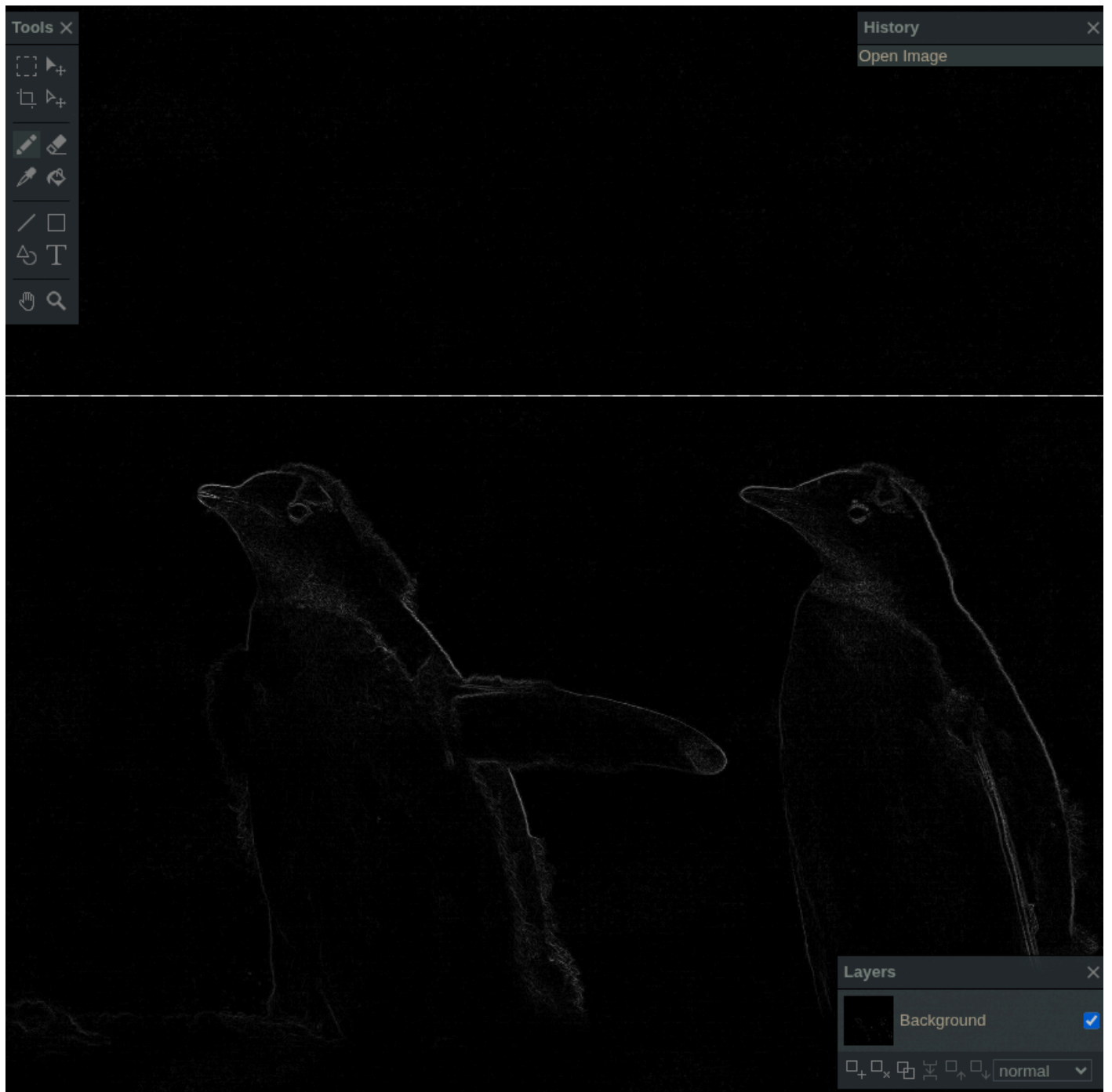


Imagen 6000x6000

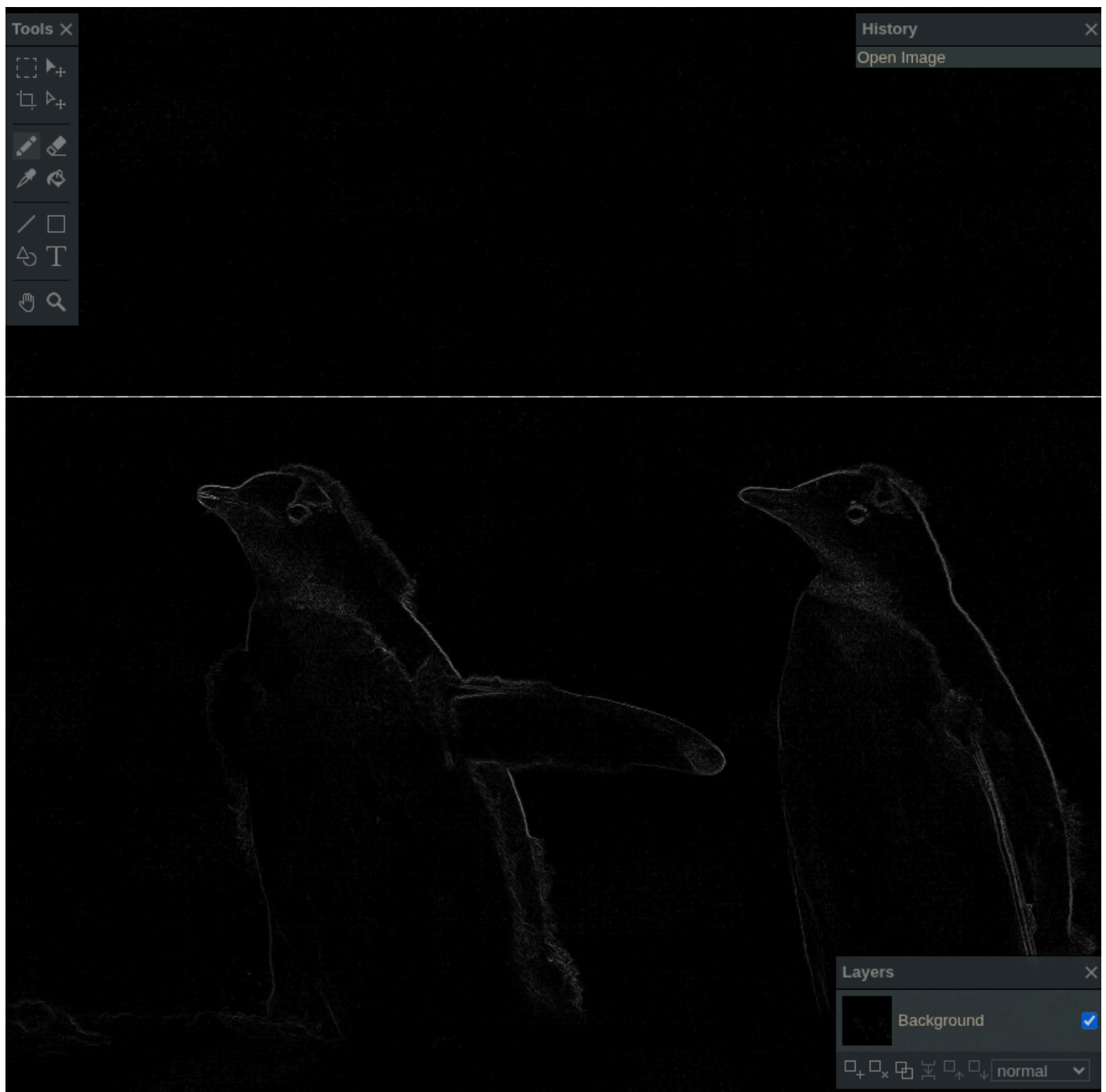
Imagen original:



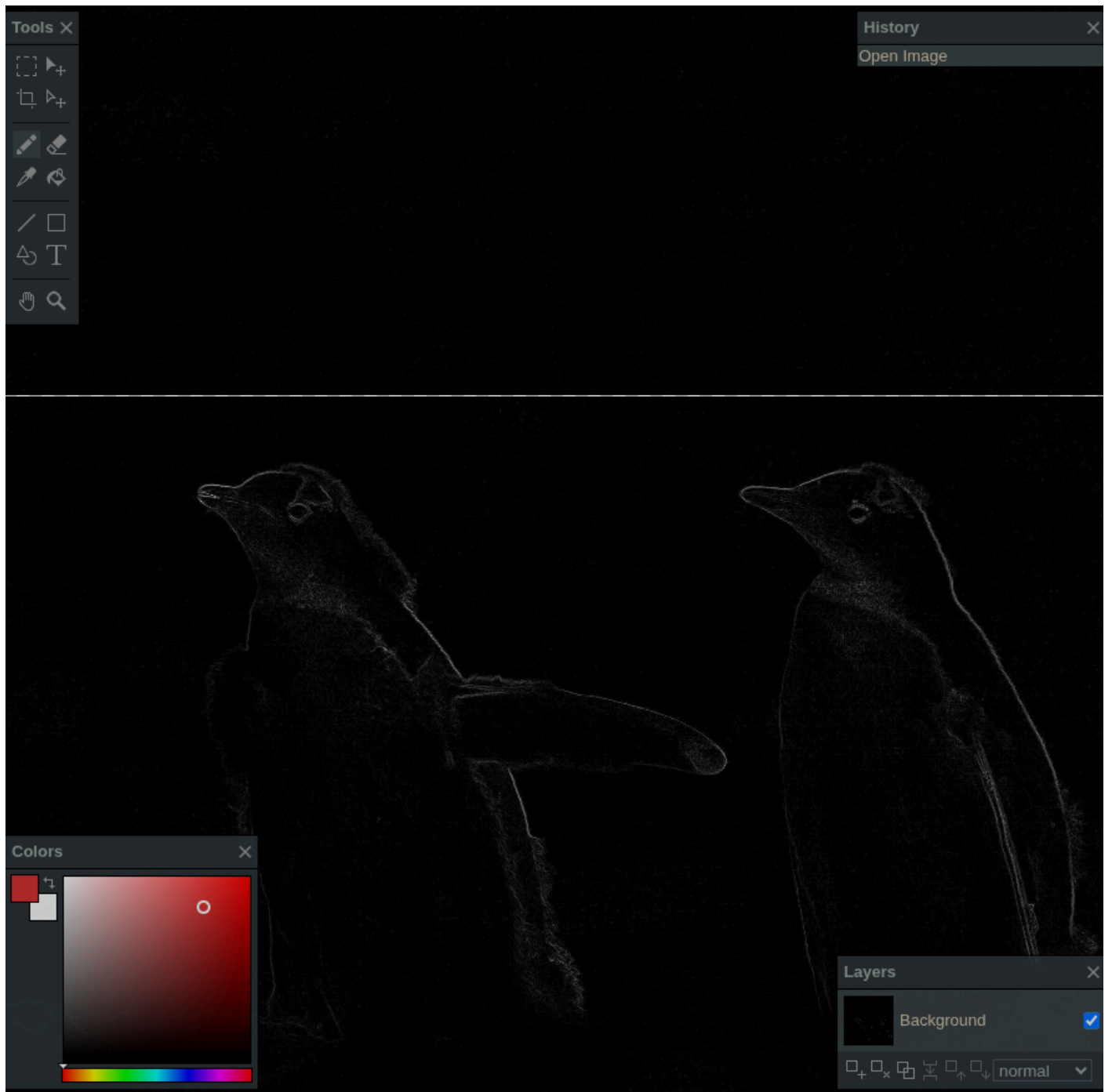
Salida Sobel secuencial:



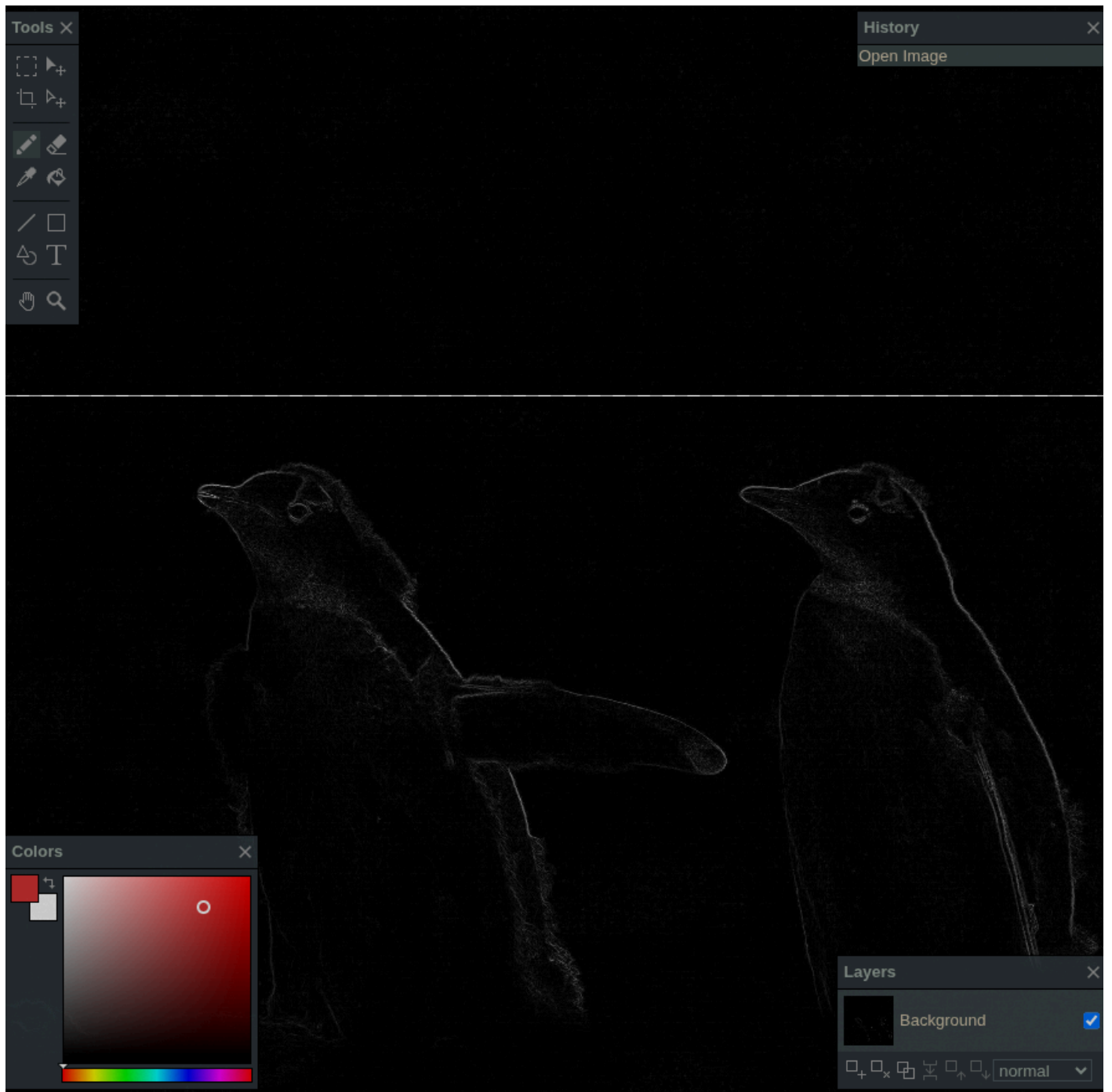
Salida Sobel NumPy:



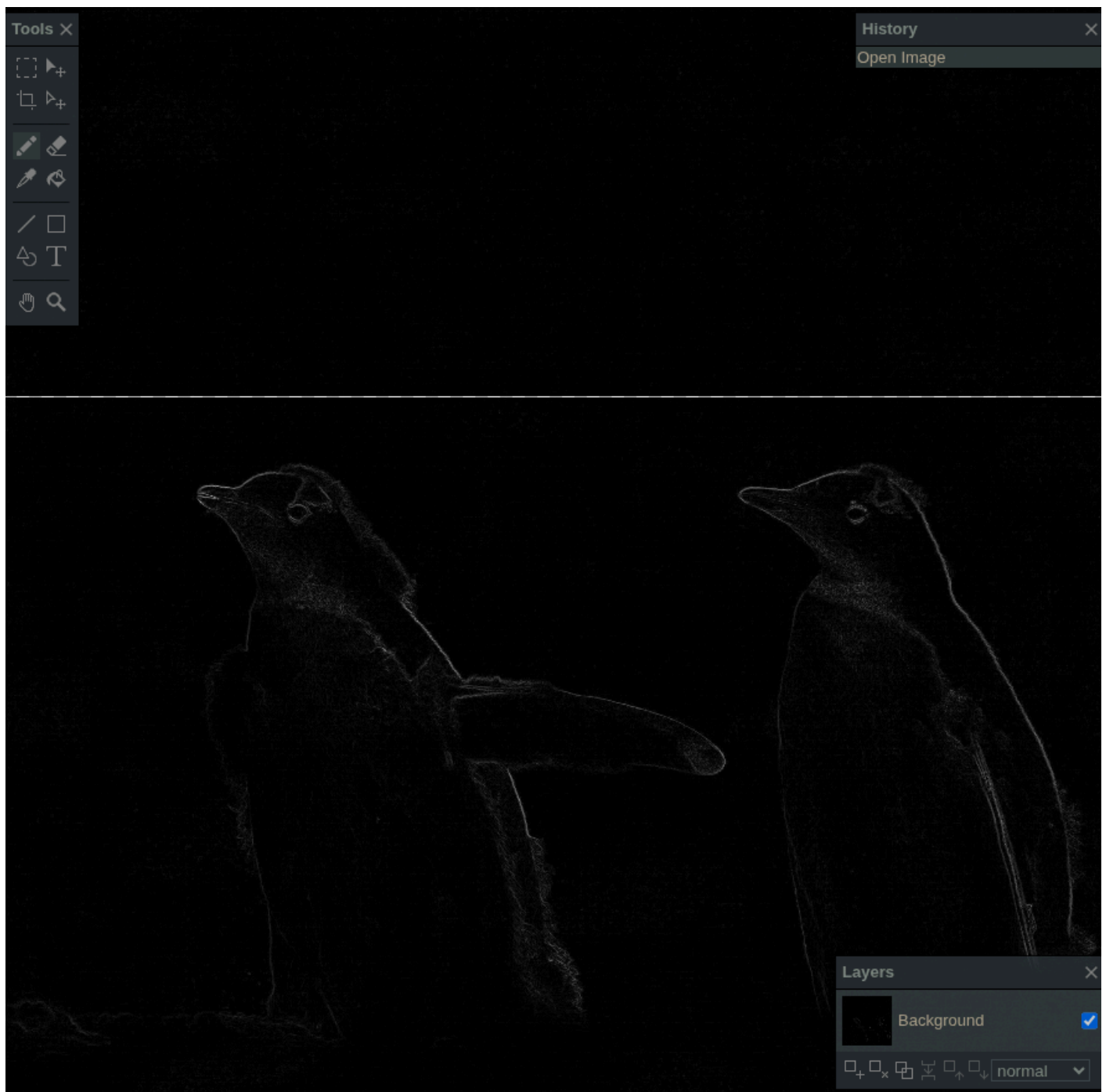
Salida Sobel Numba CPU:



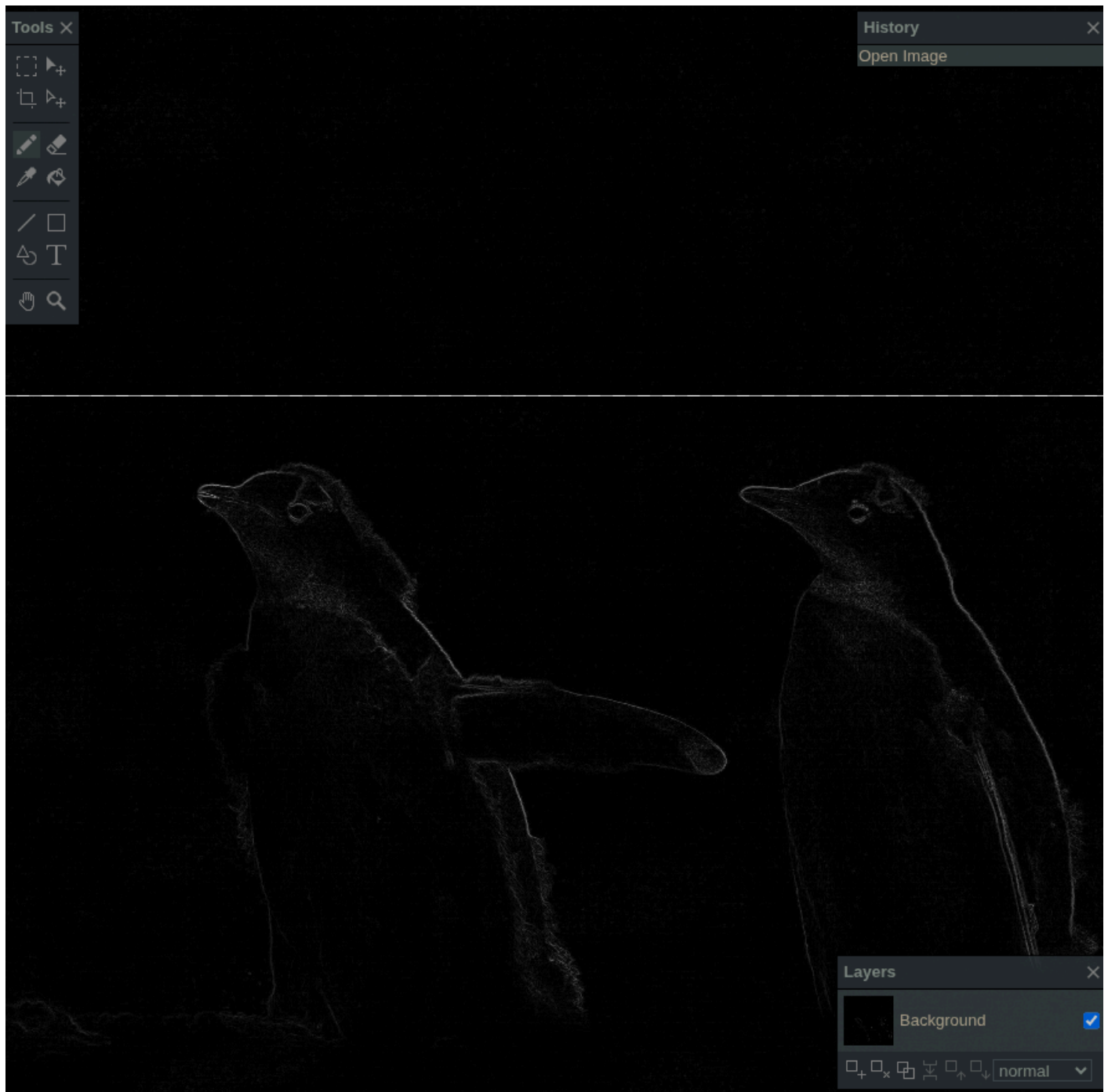
Salida Sobel Numba GPU:



Salida Sobel PyTorch CPU:



Salida Sobel PyTorch GPU:



4. Análisis

4.1 Entrega 1: secuencial, NumPy y Numba paralelo CPU

La primera entrega permite ver el salto entre tres formas de expresar el mismo cálculo en CPU. La versión secuencial es el punto de referencia y queda rápidamente limitada por el costo de recorrer píxel por píxel desde Python. En 750x750 tarda 0.52065 s y en 6000x6000 llega a 33.28282 s. El crecimiento es esperable: al duplicar la dimensión de la imagen, la

cantidad de píxeles se cuadruplica, y el bucle Python paga ese costo de forma directa.

NumPy mejora mucho respecto del secuencial porque elimina los bucles explícitos de Python y delega el trabajo a operaciones vectorizadas. Esa mejora es clara en todos los tamaños, aunque su speed-up baja de 12.60x en 750x750 a 4.50x en 6000x6000. La caída indica que, para imágenes grandes, el costo de crear arreglos intermedios y mover memoria empieza a pesar más que el beneficio de la vectorización.

Numba CPU es el método más sólido dentro de esta entrega. Al compilar los bucles y paralelizar por filas con 4 núcleos físicos, mantiene una estructura cercana al algoritmo original pero evita el overhead del intérprete. Su speed-up crece de 45.60x en 750x750 a 546.19x en 6000x6000. Esto muestra que el costo fijo de compilar y coordinar trabajo se amortiza mejor cuando aumenta el tamaño de imagen.

En términos de salida, los tres métodos producen el mismo porcentaje de píxeles blancos para cada tamaño. Por lo tanto, la diferencia observada no se debe a variaciones del filtro, sino al modo en que cada implementación ejecuta la misma operación.

4.2 Entrega 2: Numba GPU

La segunda entrega incorpora GPU mediante Numba CUDA. Esta versión sigue un modelo más cercano al hardware: se define una grilla de bloques, cada hilo procesa un píxel y se separan explícitamente la transferencia de datos y los kernels. El resultado global es el mejor del trabajo: Numba GPU obtiene el menor tiempo total en los cuatro tamaños.

Frente a Numba CPU, la GPU gana en todos los casos, pero la ventaja no crece de forma lineal con la resolución. En 750x750 la mejora con transferencias incluidas es de 6.37x; en 1500x1500 baja a 1.86x; en 3000x3000 queda en 1.30x; y en 6000x6000 en 1.25x. Esto no significa que la GPU escale mal, sino que Numba CPU ya es un baseline muy optimizado. La comparación contra secuencial muestra otra lectura: el speed-up de Numba GPU crece de 290.44x a 684.92x al pasar de 750x750 a 6000x6000.

El costo de transferencia CPU $\longleftrightarrow$ GPU se amortiza en todos los tamaños medidos porque el tiempo total de Numba GPU siempre queda por debajo de Numba CPU. Aun así, las transferencias no son despreciables. En 6000x6000, por ejemplo, mover datos suma 0.01758 s sobre un total de 0.04859 s. La GPU gana porque el cómputo de los kernels es muy bajo, pero una parte

importante del tiempo total no corresponde al filtro en sí sino al movimiento de datos.

La tendencia muestra un umbral conceptual importante: comparar contra secuencial evidencia el beneficio masivo de paralelizar el problema; comparar contra Numba CPU evidencia el costo real de competir contra una CPU ya compilada y paralelizada. Por eso, Numba GPU es el mejor método global, pero la distancia frente a Numba CPU se vuelve ajustada en los tamaños grandes.

4.3 Entrega 3: PyTorch CPU y PyTorch GPU

La tercera entrega cambia el nivel de abstracción. PyTorch permite expresar el problema con tensores y ejecutar la misma lógica en CPU o GPU. En 750x750, PyTorch GPU no logra superar a PyTorch CPU: tarda 0.01659 s contra 0.00941 s. En ese tamaño, la sobrecarga de usar GPU y el costo de las operaciones internas no se compensan.

A partir de 1500x1500, PyTorch GPU sí muestra una ventaja clara sobre PyTorch CPU: 8.18x, 8.42x y 9.22x para 1500x1500, 3000x3000 y 6000x6000. Este comportamiento coincide con el criterio del libro: una GPU no es automáticamente más rápida; empieza a convenir cuando el volumen de trabajo permite amortizar transferencias y overhead de ejecución.

Frente a las mejores implementaciones anteriores, PyTorch GPU queda detrás de Numba GPU. La diferencia no es enorme en los tamaños medianos y grandes, pero existe: PyTorch GPU queda 1.46x más lento que Numba GPU en 1500x1500, 1.74x en 3000x3000 y 1.85x en 6000x6000. Esto es razonable porque Numba CUDA está escrito como kernel específico para este problema, mientras que PyTorch usa una abstracción más general basada en tensores y `conv2d`.

Las salidas de PyTorch son consistentes con las de NumPy y Numba según el porcentaje de píxeles blancos. Si existieran diferencias en otra ejecución, los factores más probables serían redondeo al convertir de `float` a `uint8`, manejo de bordes, precisión numérica o una variación en la fórmula usada para combinar `Gx` y `Gy`.

4.4 Lectura global

El caso secuencial cumple el rol de baseline para todo el trabajo. A partir de ese punto, cada técnica muestra una idea distinta de optimización: NumPy cambia el nivel de abstracción y vectoriza; Numba CPU

conserva la forma de bucles pero compila y paraleliza; Numba GPU expone el modelo CUDA; PyTorch permite trabajar con tensores y mover el cálculo entre CPU y GPU con menor control explícito.

La mejor implementación en esta medición fue Numba GPU. Sin embargo, el resultado más interesante no es solo que GPU sea rápida, sino cómo cambia la comparación según el baseline. Contra secuencial, todos los métodos optimizados parecen extremadamente superiores. Contra Numba CPU, la ventaja de GPU existe pero debe justificar transferencias y overhead. Contra PyTorch CPU, PyTorch GPU muestra que el acelerador empieza a tener sentido recién cuando el problema supera cierto tamaño. En conjunto, los resultados refuerzan la idea central de la materia: paralelizar no es solamente usar más hardware, sino elegir una forma de expresar el cálculo que se ajuste al tamaño del problema y al costo de mover datos.

5. Conclusiones

Todas las implementaciones respetan el mismo criterio matemático de Sobel y producen resultados consistentes según el porcentaje de píxeles blancos. Esto permite comparar rendimiento sin introducir diferencias de salida.

El método secuencial es el caso base y confirma el costo alto de recorrer píxeles en Python puro. NumPy mejora el tiempo mediante vectorización, pero Numba CPU logra una mejora mayor al compilar los bucles y paralelizar por filas con 4 workers físicos. En GPU, Numba CUDA fue el mejor método en todos los tamaños, con tiempos totales de 0.00179 s, 0.00490 s, 0.01509 s y 0.04859 s para 750x750, 1500x1500, 3000x3000 y 6000x6000 respectivamente.

PyTorch GPU mostró una mejora clara frente a PyTorch CPU desde 1500x1500 en adelante, pero no superó a Numba GPU. Esto sugiere que PyTorch es conveniente cuando el problema ya está formulado como tensores o forma parte de una cadena de operaciones en GPU, mientras que Numba CUDA permite una implementación más directa y eficiente para este filtro puntual.

Como conclusión general, el mejor resultado del trabajo fue Numba GPU. Sin embargo, Numba CPU también resulta muy competitivo, especialmente

considerando que usa solamente los 4 núcleos físicos del procesador y evita el costo de transferencias CPU $\longleftrightarrow$ GPU.